

CAKE requires programming - On the provable post-quantum security of (O)CAKE

Kathrin Hövelmanns¹ , Andreas Hülsing^{1,2} , Mikhail Kudinov¹ , and Silvia Ritsch¹ 

¹ Eindhoven University of Technology, The Netherlands

² SandboxAQ, Paolo Alto, US

Abstract. In this work we revisit the post-quantum security of KEM-based password-authenticated key exchange (PAKE), specifically that of (O)CAKE. So far, these schemes evaded a security proof considering quantum adversaries. We give a detailed analysis of why this is the case, determining the missing proof techniques. To this end, we first provide a proof of security in the post-quantum setting, up to a single gap. This proof already turns out to be technically involved, requiring advanced techniques to reason in the QROM, including the compressed oracle and the extractable QROM. To pave the way towards closing the gap, we then further identify a way to model the ideal cipher by means of a simulator that enables a proof. The simulator provides certain programming abilities, which prove to be both a sufficient and necessary condition to close the proof gap: it is sufficient since we complete our proof, using the simulator, and necessary in many relevant cases – we give a meta-reduction based on KEM anonymity. The meta-reduction shows the impossibility of a non-programming reduction that covers a class of KEMs including Kyber / ML-KEM.

Keywords: Post-quantum cryptography, password-based authenticated key exchange, PAKE, CAKE, OCAKE, quantum ideal-cipher model, QIC, QROM, meta reduction.

1 Introduction

Passwords are still one of the most common means of user authentication. The reason is that they are easy to use on different devices, without the requirement for additional hardware. At the same time, passwords are a common reason for

K.H. was supported by an NWO VENI grant (Project No. VI.Veni.222.397). A.H. and M.K. were supported by an NWO VIDI grant (Project No. VI.Vidi.193.066). S.R. is part of the Quantum-Safe Internet (QSI) ITN which received funding from the European Union’s Horizon-Europe programme as Marie Skłodowska-Curie Action (PROJECT 101072637 - HORIZON -MSCA-2021-DN-01). The authors would like to thank Manuel Barbosa, Anne Broadbent, Christian Majenz, and Marc Vostermans for helpful discussions.

security breaches: They generally have low entropy, making them comparatively easy to guess, and as they have to be transmitted, they can potentially be intercepted. A way to avoid issues caused by interception is password-authenticated key exchange (PAKE). In a PAKE, the password is not transmitted but is used by communicating parties in a protocol to derive a shared session key. PAKE can not only be used for authentication on the Internet. It is a general tool for session authentication when only a low entropy secret is shared. This is why among the most popular applications of PAKE there is not only authentication in WiFi networks, but also authentication of contact-less connections to identity documents. For the latter use-case, the (low-entropy) machine-readable section of the document is read and used as a password in a PAKE (specifically PACE for ICAO passports).

This motivated major efforts put on the design of PAKE in recent years, including a competition within IETF’s Crypto-Forum Research Group (CFRG) to select new PAKE. However, most of these designs are inherently linked to group-based cryptography, as they exploit its rich algebraic structure. This renders them vulnerable to attacks by future quantum computers. While this is not as big of an issue when just thinking about the authentication part, as PAKE are generally used in short-lived authentication decisions, it still is an issue in two dimensions. First, as PAKE are used to establish session keys, also the data later secured via these keys is susceptible to store-now-decrypt-later attacks. Second, given their use in long-lived identity documents and the specification and standardization time of such, even if we start transitioning now, we will have documents with old schemes in the field for the next 15-20 years. Hence, it is necessary to start looking for alternatives to classically secure PAKE now.

Indeed, there have been several works on the topic of post-quantum PAKE. In general, these can be split into two categories: First, there are proposals that build PAKE directly from specific hardness assumptions [DAL⁺17, SA23, DKBY24, TY19, SH19]. Second, there are generic proposals [BCP⁺23, PZ23] that can construct PAKE from generic Key Encapsulation Mechanisms (KEMs) which fulfill certain somewhat-standard properties ([Xag22, GMP22, MX23]). The former are mostly based on specific hardness assumptions on lattices and some come with a security proof. Some even come with a security proof that considers quantum adversaries[LLH24]. These proposals are usually quite efficient, as they can exploit the available algebraic structure. The latter do not have access to this structure, which can render them a bit less efficient. However, they have the advantage that they can be instantiated with a wide range of KEM. Given the experience that quantum computers are able to break virtually all traditional public key cryptography due their use of very few, closely related hardness assumptions, this freedom of choosing an arbitrary KEM, independent of the underlying hardness assumption, seems like a strong advantage in practice. For that reason, we study this latter category in this work. A comprehensive overview over post-quantum PAKE proposals and the models used in their analysis is given in [AHMW25], Table 1.

The main contender of this category are the recently proposed protocols CAKE & OCAKE [BCP⁺23]. These are instances of the well-known Encrypted Key Exchange (EKE) blueprint that was also used in PACE, SPEKE, and many more. It has been demonstrated, in the UC-model [BCP⁺23] as well as in the game-based setting [PZ23, AHHR24], that CAKE & OCAKE are secure against attacks using classical computers as long as the used KEM provides three properties: It has to achieve indistinguishable keys under weak attacks (IND-CPA or IND-PCA); it has to guarantee anonymity of ciphertexts (ANO-CPA or ANO-PCA), meaning one cannot infer the used public key when seeing a ciphertext; and finally it has to guarantee that public keys are indistinguishable from random bit strings. However, a proof of security against quantum attackers is so far still missing.

KEM-EKE and its security. (O)CAKE is essentially a version of EKE instantiated using a KEM. We here consider OCAKE. (CAKE only differs in minor aspects.) The protocol initiator **I** and responder **R** share a password pw . The initiator generates an ephemeral KEM key pair pk, sk . Then it uses the password to compute an encrypted public key $apk \leftarrow \text{BC}(\text{KDF}(pw), pk)$ using a block cipher BC and sends apk to the responder. The responder recovers $pk \leftarrow \text{BC}^{-1}(\text{KDF}(pw), apk)$ using the password, and computes an encapsulation $(c, k) \leftarrow \text{Encap}(pk)$. It sends c , potentially accompanied by a key-confirmation tag, back to **I** who obtains the shared key via $k \leftarrow \text{Decap}(sk, c)$. This is potentially followed by another key confirmation message from **I** to **R**.

A key property of PAKE is that adversarial observation of the exchanged messages must not admit offline password-guessing attacks, and that active attacks only allow to verify at most a single password guess. This is necessary due to the commonly limited size of practically used password spaces. For OCAKE this means – among others – that the second message must be computationally independent of the first. This is where KEM anonymity is required. Consider the following attack: A malicious initiator $\mathcal{A}$ sends a random apk to an honest **R**. In addition, $\mathcal{A}$ computes $pk_i \leftarrow \text{BC}^{-1}(\text{KDF}(pw_i), apk)$ for all passwords pw_i in the password space. This is possible due to the limited size of the password space. When receiving back the ciphertext c , the adversary simply checks under which pk_i the ciphertext c is valid and extracts the respective password this way. If the KEM provides no anonymity, this attack is possible³.

Consequently, if we want to base security on a computationally anonymous KEM, a reduction has to be able to use this very adversary to solve an anonymity challenge. In the formal definition of anonymity we are given two public keys and a ciphertext that is an encapsulation under one of the two. The task is to decide which of the two public keys was used to create the ciphertext. In the classical proof, the respective reduction simulates the ideal cipher via lazy sampling and adaptive programming. The latter essentially ensures that under the correct password, any apk sent by $\mathcal{A}$ during an active attack decrypts to one of the challenge public keys for the anonymity game. We can then embed

³ This attack is a slight simplification of the actual attack which is more complex due to the way anonymity is defined.

the challenge ciphertext as the second message. If $\mathcal{A}$ succeeds, we know that c was likely generated with the programmed public key. If $\mathcal{A}$ fails, c was likely generated using the other public key.

The quantum proof gap. When moving to a setting that considers quantum adversaries, it has been known since [BDF⁺11] that one has to provide such adversaries with *quantum* access to random oracles and ideal ciphers. For many proof steps that involve random oracles, this makes it necessary to revisit the used arguments. Whereas in the classical setting, such steps often used comparably simple ‘standard’ arguments, allowing quantum access can render those steps significantly more involved. (Examples are query inspection, adaptive reprogramming, or online extraction.) While this can be challenging, the community has made huge progress towards dealing with these changes for random oracles. When it comes to ideal ciphers, however, the situation is different. While this area of research is very active [Hos25, CHL⁺25, ACMT25, CP25, MMW24, Unr23a, Unr21, CMSZ19], at this point advanced techniques such as efficiently simulating ideal ciphers with adaptive reprogramming still seem out of reach.

In short, challenging proofs compound with a need for techniques that are missing entirely. This might explain why OCAKE / CAKE has no security proof against quantum adversaries, yet. In this work we thus set out to understand how to surpass this barrier, by determining what exactly is lacking. Moreover, given that researchers try to circumvent the ideal cipher model altogether, we are interested in potential avenues to do so (and what paths are hopeless).

Our contribution In this work, we determine the necessary & sufficient (missing) ingredient for a reductionist proof to succeed, when considering quantum adversaries. Towards this end, we provide a proof against quantum adversaries in the quantum-accessible random oracle and ideal cipher model (QROM & QIC), assuming the existence of an efficient simulator for the ideal cipher that is capable of a certain form of reprogramming. We give a precise definition of this simulator and also demonstrate necessity of such a simulator when constructing an EKE PAKE from arbitrary KEM. Note that for the necessity argument it is sufficient to point out a single KEM for which a security proof cannot succeed without implementing such a simulator.

Indeed, we show that we find such KEM in all KEM that follow the full Fujisaki-Okamoto (FO) KEM blueprint [HHK17, FO99, FO13], assuming they satisfy typical properties (correctness and spreadness) and a very weak form of robustness. (More concretely, we require a very weak robustness property from the version of the considered KEM that rejects explicitly, while the considered KEM may well be implicitly rejecting.)

This result applies to the new NIST standard ML-KEM, also known as Kyber, and we conjecture that it also applies to most other post-quantum KEM that were considered in the NIST competition. Since OCAKE can only achieve post-quantum security when instantiated with a post-quantum KEM, we believe that our result covers the most relevant cases.

On a technical level, we show that CAKE/OCAKE can be broken by an adversary that can break anonymity of the used KEM. Consequently, any security

proof that works for general KEM has to include a reduction from anonymity. We give a meta-reduction which shows that any such reduction – if it works for the aforementioned class of KEMs – has to implement this programming simulator. We show that otherwise, the reduction must be able to break anonymity itself. Our result is limited to the aforementioned KEM class because it allows a simulation of a successful adversary that just uses random oracle techniques, avoiding tricks like rewinding or the use of the generic group model.

The main results can be summarized by two (informal variants of the main) theorems:

Theorem 1 (Protocol OCAKE is secure in the QROM under SIM.). *Protocol OCAKE is secure for any KEM that is $(1 - \delta_{\text{KEM}})$ -correct, multi-user-IND-CPA- and multi-user-ANO-PCA-secure and has uniform public keys, provided the existence of a programming simulator SIM.*

Theorem 2 (Any generic anonymity reduction must program.). *There cannot exist any reduction $\mathcal{R}$ that reduces ANO-PCA to BPR security of OCAKE generically for any KEM without implementing a programming simulator SIM.*

Feasibility of instantiating the simulation. There has been some progress on simulations for random permutations that can support query-recording [Unr23b, MMW24, ACMT25], but to the best of our knowledge there is currently none that can fulfill the programmability requirement we isolate. In fact, it may prove impossible to achieve for an ideal cipher. While we phrase our results in most places with respect to an ideal cipher for ease of exposition, they can be easily generalized to families of bijections where each bijection is sampled from an independent distribution over the set of bijections.

Taking this generality into account, the programming requirement outlined in this paper immediately removes some constructions that try to avoid the QIC from consideration. One such example is a masking-based approach where the password (and potentially a session identifier) is hashed and used as an xor-*mask* for the public key $apk = pk \oplus H(pw, sid)$. Since many efficient instantiations are ruled out by the requirement, the results of this paper indicate that it is worth investigating more complex instantiations that may come with a higher cost. We hope that our result can enable research into possible instantiations of the programming simulator we define, thereby leading the way to generic post-quantum PAKE from KEM with security against quantum adversaries.

Organization. The organization of this paper is depicted in Fig. 1

2 Preliminaries

In this section, we recall some notation and/or security notions for block ciphers, key encapsulation mechanisms, and hash functions, as well as the CAKE/OCAKE protocol.

Notation. We denote deterministic output y of an algorithm A on input x by $y := A(x)$. We denote algorithms with access to an oracle $\mathcal{O}$ by $A^{\mathcal{O}}$. Unless

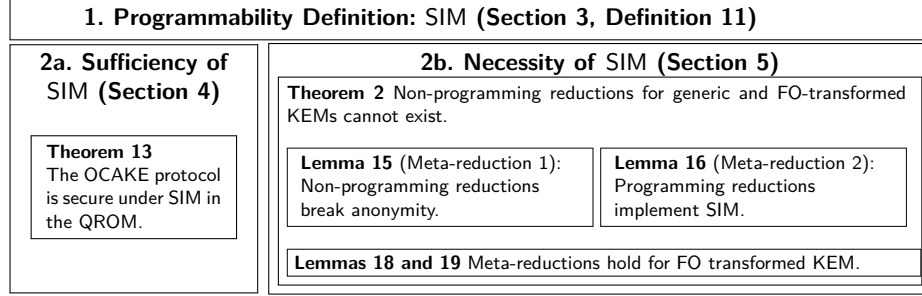


Fig. 1. Organization of this paper. We introduce the programmability requirement by defining a simulator in Section 3. We show sufficiency of this programmability property in Section 4, by adapting OCAKE’s security proof (in the Bellare-Pointcheval-Rogaway model) to the quantum random oracle model. Lastly, we prove necessity of the programmability property in Section 5, using a meta-reduction technique. (We show that the meta-reductions hold for KEMs that follow the Fujisaki-Okamoto (FO) blueprint.)

stated otherwise, we assume all our algorithms to be probabilistic and denote the computation by $y \leftarrow \$ A(x)$.

Block Ciphers The proof of security of the OCAKE protocol is given in the ideal cipher model [Bla06]. Analogous to the ROM for hash functions, the ideal cipher (IC) is an idealized description of a block cipher.

Definition 3 (Block Cipher (BC)). A block cipher of block length n and key length k consists of two algorithms $BC : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ and $BC^{-1} : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ such that for every plaintext $m \in \{0, 1\}^n$ and key $k \in \{0, 1\}^k$, decryption undoes encryption: $BC^{-1}(k, BC(k, m)) = m$.

Definition 4 (Ideal Cipher (IC)). An ideal cipher is a collection of random permutations indexed by a key, to which all parties (including the adversary) are given oracle access. I.e., it is a pair of random functions $E, D : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{M}$, such that $D(k, E(k, m)) = m$ and $E(k, D(k, m)) = m$ for all k, m in $\mathcal{K} \times \mathcal{M}$.

Key Encapsulation Mechanisms (KEM). We start with the functional definition of KEMs. Afterwards we discuss their security. In the end, we introduce a new robustness notion.

Definition 5 (Key Encapsulation Mechanisms (KEMs)). A KEM is a triple of algorithms $KEM = (KGen, Encap, Decap)$, together with a public key space $\mathcal{PK}$ and secret key space $\mathcal{SK}$.

- $KGen \rightarrow (pk, sk)$: On empty input probabilistically **return** key pair (pk, sk) , where pk also defines a finite key space $\mathcal{K}$ and a ciphertext space $\mathcal{C}$.
- $Encap(pk) \rightarrow (c, K)$: On input pk probabilistically **return** a pair $(K, c) \in \mathcal{K} \times \mathcal{C}$. We call c the encapsulation of the key K .
- $Decap(sk, c) \rightarrow K$: On input sk and ciphertext c deterministically **return** a key $K \in \mathcal{K}$.

Game ANO-PCA ^b ($\mathcal{A}$)	1-PC0(c, K)
01 $(pk_0, sk_0) \leftarrow \KGen	06 if $c \neq c^*$
02 $(pk_1, sk_1) \leftarrow \KGen	07 $K' \leftarrow \text{Decap}(sk_0, c)$
03 $(c^*, K^*) \leftarrow \text{Encap}(pk_b)$	08 return $\llbracket K = K' \rrbracket$
04 $b' \leftarrow \mathcal{A}^{1\text{-PC0}}(pk_0, pk_1, c^*)$	09 else return $\perp$
05 return $\llbracket b = b' \rrbracket$	

Fig. 2. The security game for anonymity (ANO-PCA) of KEM. Oracle 1-PC0 can be queried at most once.

Game $(n, q_C) - \text{IND-CPA}^b$	Game $(n, q_C) - \text{ANO-PCA}^b(\mathcal{A})$	Chall _{q_C} ^b (j)
01 for $i \in [n]$	01 for $j \in [n]$	08 $(c_b, K_b) \leftarrow \$\text{Encap}(pk_{b,j})$
02 $(pk_i, sk_i) \leftarrow \KGen	02 $(pk_{0,j}, sk_{0,j}) \leftarrow \KGen	09 $\mathcal{L}_j^c \leftarrow \mathcal{L}_j^c \cup \{c_b\}$
03 $pk_i.\text{append}(pk_i)$	03 $(pk_{1,j}, sk_{1,j}) \leftarrow \KGen	10 return (c_b, K_b)
04 $b' \leftarrow \mathcal{A}^{\text{Chall}_{q_C}^b}(pk_i)$	04 $\vec{pk}_0.\text{append}(pk_{0,j})$	
05 return b'	05 $\vec{pk}_1.\text{append}(pk_{1,j})$	1-PC0(j, c, K) // once per j , for $c \notin \mathcal{L}_j^c$
Chall _{q_C} ^b (j)	06 $b' \leftarrow \mathcal{A}^{1\text{-PC0}, \text{Chall}_{q_C}^b}(\vec{pk}_0, \vec{pk}_1)$	11 $K' \leftarrow \text{Decap}(sk_{0,j}, c)$
06 $(c, K_0) \leftarrow \$\text{Encap}(pk_j)$	07 return b'	12 return $\llbracket K = K' \rrbracket$
07 $K_1 \xleftarrow{\text{unif}} \$\mathcal{K}$		13 else return $\perp$
08 return (c, K_b)		

Fig. 3. The Multi-user games for anonymity $((n, q_C) - \text{ANO-PCA})$ and indistinguishability $((n, q_C) - \text{IND-CPA})$ for KEM, for n users. Challenge oracle **Chall** can be queried at most q_C many times per user, and 1-PC0 once per public key.

Definition 6 (δ -Correctness (average-case)). We say that KEM is average-case $(1 - \delta)$ -correct if $\Pr[\text{Decap}(sk, c) = K | (c, K) \leftarrow \$\text{Encap}(pk)] \geq 1 - \delta$, where the probability is taken over $(pk, sk) \leftarrow \text{KGen}()$ and the random coins of Encap .

Definition 7 (γ -spreadness). We say that PKE is γ -spread iff for all key pairs $(pk, sk) \in \text{supp}(\text{KGen})$ and all messages $m \in \mathcal{M}$ it holds that

$$\max_{c \in \mathcal{C}} \Pr[\text{PKE.Enc}(pk, m) = c] \leq 2^{-\gamma},$$

where the probability is taken over the internal randomness of PKE.Enc .

Definition 8 (Anonymity for KEM). Let KEM be a key encapsulation mechanism with public-key space $\mathcal{PK}$ and key space $\mathcal{K}$. We define the ANO-PCA game as depicted in Fig. 2, relative to challenge bit b , and the advantage function of an adversary $\mathcal{A}$ against KEM as

$$\text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}(\mathcal{A}) := |\Pr[\text{ANO-PCA}^0(\mathcal{A}) \Rightarrow 0] - \Pr[\text{ANO-PCA}^1(\mathcal{A}) \Rightarrow 0]|$$

Finally, we introduce an extremely weak notion of robustness that we call honest robustness. One may think of it as a form of weak robustness (WROB) [GMP22] where all values are randomly chosen instead of given by an adversary. Consequently, the notion is implied by WROB.

Definition 9 (hROB). *For any given key encapsulation mechanism KEM, we define the advantage against honest robustness as*

$$\text{Adv}_{\text{KEM}}^{\text{hROB}} \mathcal{A} := \Pr \left[\begin{array}{l} (pk_1, \cdot), (\cdot, sk_2) \leftarrow \$ \text{KGen}() \\ (c, \cdot) \leftarrow \$ \text{Encap}(pk_1) \\ k := \text{Decap}(sk_2, c) \end{array} \right]$$

Extractable Compressed Random Oracle. Here we introduce an extension of Zhandry's Compressed Random Oracle [Zha19] called extractable Compressed Random Oracle [DFMS22] (eCO). Using **eCO** represents a quantum-accessible Random Oracle with an additional mechanism that allows to check if there was a query that satisfies some function f . This gives a form of observable QROM. The **eCO** developed in [DFMS22] has two interfaces: one that mimics the behavior of a random oracle (**eCO.R0**) and another that allows extraction of information (**eCO.E**). The **eCO.R0_f** interface defined relative to a function f takes a classical value t as input and simulates a quantum measurement that "collapses" the oracle database, allowing it to yield a specific outcome x . After the measurement, the database is in a state where all the values for **eCO.R0** collapse to the values y that satisfy the equation $f(x, y) = t$. Whenever the function f used is clear from context we write **eCO.E** instead of **eCO.E_f**. To clarify, consider the classical analogue where the Random Oracle is simulated via lazy-sampling. Then for an extraction query we go through the database and output the first pair (x, y) such that x is the smallest value for which $f(x, y) = t$.

Previously, in [DFMS22, HM24] the **eCO** was used to simulate Decapsulation queries during the runtime of the adversary. In our case we use it to check if the adversarially generated tags are valid.

Now let's give a more formal treatment of the **eCO**. We will closely follow the description from [DFMS22, HM24]. The description represents an inefficient version of the **eCO** which can be made efficient using sparse encoding (c.f., [DFMS22]). The simulator **eCO** for a random function $O: \{0, 1\}^m \rightarrow \{0, 1\}^n$ is a stateful oracle with a state stored in a quantum register $D = D_{0^m} \dots D_{1^m}$, where for each input value $x \in \{0, 1\}^m$, register D_x has $n+1$ qubits used to store superpositions of n -bit output strings y , encoded as $0y$, and an additional symbol $\perp$, encoded as 10^n . We adopt the convention that an operator expecting n input qubits acts on the last n qubits when applied to one of the registers D_x . The compressed oracle has the following three components.

1. The initial state of the oracle, $|\phi\rangle = |\perp\rangle^{2^m}$
2. A quantum query with query input register X and output register Y is answered using the oracle unitary O_{XYD} defined by $O_{XYD}|X\rangle_X = |X\rangle_X \otimes (F_{D_x} \text{CNOT}_{D_x:Y}^{\otimes n} F_{D_x})$, where $F|\perp\rangle = |\phi_0\rangle$, $F|\phi_0\rangle = |\perp\rangle$ and $F|\psi\rangle = |\psi\rangle$ for all $|\psi\rangle$ such that $\langle\psi|\perp\rangle = \langle\psi|\phi_0\rangle = 0$, with $|\phi_0\rangle = |+\rangle^{\otimes n}$ being the uniform superposition.
3. A recovery algorithm that recovers a standard QRO O : apply $F^{\otimes 2^m}$ to D and measure it to obtain the function table of O .

The formal description of the extraction interface is the following. Given a random oracle $O : \{0,1\}^m \rightarrow \{0,1\}^n$, let $f : \{0,1\}^m \times \{0,1\}^n \rightarrow \{0,1\}^\ell$ be a function. We define a family of measurements $(\mathcal{M}^t)_{t \in \{0,1\}^\ell}$. The measurement $\mathcal{M}^t$ has measurement projectors $\{\Sigma^{t,x}\}_{x \in \{0,1\}^m \cup \{\emptyset\}}$.

Let $\Pi^{t,x} = \sum_{y \in \{0,1\}^n: f(x,y)=t} |y\rangle\langle y|$. Then, for $x \in \{0,1\}^m$, the projector selects the case where D_x is the first (in lexicographical order) register that contains y such

that $f(x,y) = t$, i.e. $\Sigma^{t,x} = \bigotimes_{x' < x} \bar{\Pi}_{D_{x'}}^{t,x'} \otimes \Pi_{D_x}^{t,x}$, and $\bar{\Pi} = \mathbb{1} - \Pi$. The remaining projector corresponds to the case where no register contains such a y , i.e. $\Sigma^{t,\emptyset} = \bigotimes_{x' \in \{0,1\}^m} \bar{\Pi}_{D_{x'}}^{t,x'}$. $\mathbf{eCO}$ is initialized with the initial state of the compressed oracle. $\mathbf{eCO.R0}$ is quantum-accessible, $\mathbf{eCO.E}$ is a classical oracle interface that, on input t , applies $\mathcal{M}^t$ to $\mathbf{eCO}$'s internal state. The following properties of $\mathbf{eCO}$ are required for the main lemma that will be used in our proof. We define a relationship R based on function f and a target value t as all input output pairs that satisfy function $f(x,y) = t$. $R_{f,t}(x,y) :\Leftrightarrow f(x,y) = t$. Next, let's define $\Gamma_R := \max_x |\{y \mid R(x,y)\}|$. Γ_R outputs the maximum number values y that satisfy a relationship R for some input x . And finally we define $\Gamma(f) = \max_t \Gamma_{R_{f,t}}$. $\Gamma(f)$ outputs the maximum number of outputs y that satisfy some relationship $R_{f,t}$ (maximizing over target values t) for some input x . The $\Gamma(f)$ value determines whether we are able to use extractions without disturbing the random oracle too much. This is stated in the following lemma.

Lemma 10 (Part of theorem 3.4 in [DFMS22], formulated in [HM24]). *The extractable RO simulator $\mathbf{eCO}$ described above, with interfaces $\mathbf{eCO.R0}$ and $\mathbf{eCO.E}$, satisfies the following properties.*

1. *If $\mathbf{eCO.E}$ is unused, $\mathbf{eCO}$ is perfectly indistinguishable from a random oracle.*
2. *Any two subsequent independent queries to $\mathbf{eCO.R0}$ commute. In particular, two subsequent classical $\mathbf{eCO.R0}$ -queries with the same input x give identical responses.*
3. *Any two subsequent independent queries to $\mathbf{eCO.E}$ commute. In particular, two subsequent $\mathbf{eCO.E}$ queries with the same input t give identical responses.*
4. *Any two subsequent independent queries to $\mathbf{eCO.E}$ and $\mathbf{eCO.R0}$ $8\sqrt{2\Gamma(f)/2^n}$ -almost-commute.*

In our application we will not give the adversary the extraction interface. We still restrict the adversary only to the RO queries. On the other hand the reduction is able to perform extraction queries to check if the oracle has been queried on the inputs that are interesting for us.

Transformation T [HHK17] **T** transforms an OW-CPA secure public-key encryption scheme into an OW-PCA secure one.

To a public-key encryption scheme $\text{PKE} = (\text{PKE.KGen}, \text{PKE.Enc}, \text{PKE.Dec})$ with message space $\mathcal{M}$ and randomness space $\mathcal{R}$, and random oracle $\mathbf{G}:\mathcal{M} \rightarrow \mathcal{R}$, we associate $\text{PKE}_1 = \mathbf{T}[\text{PKE}, \mathbf{G}]$. The algorithms of $\text{PKE}_1 = (\text{PKE.KGen}, \text{PKE.Enc}_1, \text{PKE.Dec}_1)$ are defined in Fig. 4. Note that Enc_1 deterministically computes the ciphertext as $c := \text{PKE.Enc}(pk, m; \mathbf{G}(m))$.

$\text{PKE.Enc}_1(pk, m)$	$\text{PKE.Dec}_1(sk, c)$
14 $c := \text{PKE.Enc}(pk, m; \mathcal{G}(m))$	16 $m' := \text{PKE.Dec}(sk, c)$
15 return c	17 if $m' = \perp$ or $\text{PKE.Enc}(pk, m'; \mathcal{G}(m')) \neq c$ return $\perp$
	18 else return m'

Fig. 4. The algorithms of the T-Transform.

Initiator I (Client)	Protocol OCAKE	Responder R (Server)
Password pw		Password pw
Public Key		
$k_{pw} \leftarrow \text{KDF}(pw)$		$k_{pw} \leftarrow \text{KDF}(pw)$
$(pk, sk) \leftarrow \$ \text{KGen}$		
$apk \leftarrow \text{BC}(k_{pw}, pk)$	$\xrightarrow{apk}$	$pk' \leftarrow \text{BC}^{-1}(k_{pw}, apk)$
Session Pre-Key		
$K' \leftarrow \text{Decap}(sk, c)$	$\xleftarrow{c}$	$(c, K) \leftarrow \$ \text{Encap}(pk')$
Key Derivation		
	$\xleftarrow{tag}$	$tag, SK \leftarrow \mathcal{H}(K, pw, apk, pk', c)$
$tag', SK' \leftarrow \mathcal{H}(K', pw, apk, pk, c)$		output SK
if $tag' = tag$		
output SK'		

Fig. 5. The OCAKE protocol, using a key encapsulation mechanism $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$, key derivation function KDF , hash function $\mathcal{H}$ and a block cipher $\text{BC} = (\text{BC}, \text{BC}^{-1})$. We require that the message space $\mathcal{M}$ of BC and the public space $\mathcal{PK}$ of KEM coincide, i.e., $\mathcal{PK} = \mathcal{M}$. Especially, every element of $\mathcal{M}$ must constitute a valid public key for KEM . This variant enables an optimization where tag and SK are computed using the same call to $\mathcal{H}$.

2.1 Protocols CAKE and OCAKE

In our analysis of which proof techniques are required for post-quantum PAKE, we focus on the OCAKE protocol, as shown in Fig. 5. In some cases, our analysis extends to the protocol variant CAKE, which we include in Section A. First, the initiator **I** generates a KEM key pair, encrypts the public key pk with a symmetric cipher, and sends the encrypted public key apk to the responder **R**. Upon receiving apk , **R** uses the password to recover pk , which it then used to compute an encapsulation c and a pre-key K . **R** sends the encapsulation c to **I**, together with a responder tag tag that can be used by **I** to confirm the key. **R** derives its session key SK' from pre-key K and the session transcript. Upon receiving c , **I** decapsulates the ciphertext to obtain a pre-key K' , which can then again be used to derive the session key. The **I** then derives the session key if key confirmation succeeds, i.e., if the received tag equals the one **I** derives from its own state. Assuming that both parties used the same password and that KEM worked correctly, **R** and **I** end up with the same session key. We made a minor conceptual modification to the previous descriptions of the OCAKE protocol:

we derive tag and SK in the same call to H . This variant constitutes a minor optimization and simplifies the analysis.

The first security proofs for protocols CAKE and OCAKE were given in [BCP⁺23] in the UC framework and only considered attackers without quantum capabilities. As a first step towards capturing security even against attackers with quantum capabilities, [PZ23] and [AHH⁺23] gave proofs in the BPR model for CAKE and OCAKE, respectively. The analysis in this work is based on these security proofs.

3 Programming Simulator SIM for (O)CAKE

We now define a simulation that is (computationally) indistinguishable from an ideal cipher, but answers queries by embedding values from a challenge set. This simulation models a block cipher that can be queried on quantum states.

We later prove that this simulator is sufficient to prove security against quantum adversaries of OCAKE in the BPR model in Section 4 by giving such a proof. Lastly, we use a meta-reduction technique([FLR⁺10, FF13, BV98]) in section 5 to study how programmability as presented by SIM is necessary for any security proof of OCAKE.

The programming simulator. We now introduce the simulator that describes the minimal necessary requirements to prove security of OCAKE. Given the current uncertainty in achieving programmability with quantum queries, this definition aims to maximize flexibility in its instantiation while clearly outlining the requirements.

Consider an adversary $\mathcal{A}$, a message space $\mathcal{M}$, and a key space $\mathcal{K}_{\text{SIM}}$. From this we define a family of bijections $E_k : \mathcal{M} \rightarrow \mathcal{M}$ indexed on a key $k \in \mathcal{K}_{\text{SIM}}$ where each bijection is independently sampled according to distribution $\mathcal{B}$. We denote the inverse of E_k as D_k . We also use $E(k, m)$ to denote $E_k(m)$.

In the case where $\mathcal{B}$ is the uniform distribution over all permutations, this definition is equivalent to that of an ideal cipher.

Definition 11 $((\delta_{\text{SIM}}, \varepsilon_{\text{SIM}})\text{--Programming Simulator SIM (single-session)})$.

Let adversary $\mathcal{A}$, message space $\mathcal{M}$, key space $\mathcal{K}_{\text{SIM}}$ and E be as above. We now define simulator SIM: $(E_{\text{SIM}}, D_{\text{SIM}})$ relative to a challenge set $\mathcal{X} \subset \mathcal{M}$ of non-repeating, independently and uniformly random elements of the message space $\mathcal{M}$, and a key K for each session sid , that is sampled uniformly at random from the key space at the beginning of the game. We say that SIM is a $(\delta_{\text{SIM}}, \varepsilon_{\text{SIM}})\text{--programming simulator}$ whenever we have for any adversary $\mathcal{A}$ that interacts with the oracles and eventually outputs some value apk and a session ID that:

1. SIM is indistinguishable from family E up to some failure probability ε_{SIM} , i.e., the success probability of any distinguishing adversary $\mathcal{A}_{\text{dist}}$ that wins if it outputs 0 when interacting with E, D and 1 when interacting with SIM $(\mathcal{X})$ is bounded by ε_{SIM} :

$$|\Pr[\mathcal{A}_{\text{dist}}^{E, D} \Rightarrow 0] - \Pr[\mathcal{A}_{\text{dist}}^{\text{SIM}(\mathcal{X})} \Rightarrow 0]| \leq \varepsilon_{\text{SIM}}.$$

2. SIM ensures that apk 's decryption is in the challenge set $\mathcal{X}$ with probability $1 - \delta_{\text{SIM}}$:

$$\Pr[(apk, sid) \leftarrow \mathcal{A}^{\text{SIM}(\mathcal{X})} : D_{\text{SIM}}(K[sid], apk) \in \mathcal{X}] = 1 - \delta_{\text{SIM}}.$$

where $K[sid]$ is the key associated with session sid .

Generalization to multiple sessions We define the multi-session variant of SIM analogously to the single-session version, except the adversary now outputs n_s pairs (apk_i, sid_i) . Since in a reduction, it could be that the attacker selects any of these values to attack, it is necessary that all n decryptions are in the challenge set: $\forall (apk_i, sid_i)$ output by $\mathcal{A}$, we require that $D_{k(sid_i)}(apk_i) \in \mathcal{X}$ with probability $1 - \delta_{\text{SIM}}$.

Definition 12 $((n, \delta_{\text{SIM}}, \varepsilon_{\text{SIM}})$ –**Programming Simulator SIM (multi-session)**).

Let adversary $\mathcal{A}$, message space $\mathcal{M}$, key space $\mathcal{K}_{\text{SIM}}$ and $\mathbf{E}$ be as above. We now define simulator SIM : $(\mathbf{E}_{\text{SIM}}, D_{\text{SIM}})$ relative to a challenge set $\mathcal{X} \subset \mathcal{M}$ of non-repeating, independently and uniformly random elements of the message space $\mathcal{M}$. We say that SIM is a $(\delta_{\text{SIM}}, \varepsilon_{\text{SIM}})$ –programming simulator whenever we have for any adversary $\mathcal{A}$ that interacts with the oracles and eventually outputs some values apk_i and a session IDs sid_i , $i \in \{0, 1, \dots, n-1\}$ that:

1. SIM is indistinguishable from family $\mathbf{E}$ up to some failure probability ε_{SIM} , i.e., the success probability of any distinguishing adversary $\mathcal{A}_{\text{dist}}$ that wins if it outputs 0 when interacting with $\mathbf{E}, D$ and 1 when interacting with $\text{SIM}(\mathcal{X})$ is bounded by ε_{SIM} : $|\Pr[\mathcal{A}_{\text{dist}}^{\mathbf{E}, D} \Rightarrow 0] - \Pr[\mathcal{A}_{\text{dist}}^{\text{SIM}(\mathcal{X})} \Rightarrow 0]| \leq \varepsilon_{\text{SIM}}$.
2. SIM ensures that the decryption of all apk_i is in the challenge set $\mathcal{X}$ with probability $1 - \delta_{\text{SIM}}$:

$$\Pr[(\vec{apk}, \vec{sid}) \leftarrow \mathcal{A}^{\text{SIM}(\mathcal{X})} :$$

$$\forall (sid_i, apk_i) \in (\vec{sid}, \vec{apk}) : D_{\text{SIM}}(K[sid_i], apk_i) \in \mathcal{X}] \geq (1 - \delta_{\text{SIM}}).$$

where $K[sid]$ is the key associated with session sid .

4 Security of OCAKE in the QROM

To prove security against quantum attackers, it is necessary to consider the quantum random oracle model as well as a quantum-accessible model for the symmetric cipher. We show that the simulation described in section 3 is sufficient to prove security of PAKE in the BPR model, using a generic KEM, by providing an updated security statement for OCAKE (Section 4.1). We also provide a high-level intuition for how to update the proof for CAKE (Section A.1).

4.1 Security of OCAKE under SIM

Security of the OCAKE protocol has been shown in the universal composability framework [BCP⁺23] as well as in the BPR model [AHH⁺23]. Both results use

the ROM and the ICM. (Appendix 14 gives a more thorough analysis of the role of the IC in the proof and its relation to attacks.) Building on the proof given in [AHH⁺23], we give a security proof for the OCAKE protocol in the BPR model, using the QROM and assuming the existence of a simulator SIM that effectively replaces the IC.

To lift the classical proof to the quantum setting, all *public* functions – that is, those that can be computed by the attacker – need to be modeled to allow quantum queries [BDF⁺11]. Therefore, any proof argument that considers hash functions, key derivation functions or block ciphers needs adapting to quantum queries. In this setting, we now give our new security theorem.

Theorem 13 (Tight security of OCAKE in the QROM from multi-user security of KEM and SIM). *Let KEM be a key encapsulation mechanism that is $(1 - \delta_{\text{KEM}})$ -correct, let KDF, and H be modeled as quantum random oracles with codomains $\mathcal{K}_{pw}$ and $\mathcal{T} \times SK$, BC be a quantumly-accessible block cipher that we model according to SIM with parameters δ_{SIM} and ε_{SIM} , and let $\mathcal{A}$ be a BPR adversary against OCAKE[KEM, KDF, H, BC], issuing at most n_a many *Send* queries (active attacks), n_p many *Execute* queries (passive attacks), and q_{RO} many queries to its respective random oracles. Let $n_s := n_a + n_p$ be the total number of sessions. Then there exist a multi-user-IND-CPA adversary $\mathcal{B}^I$ and a multi-user-ANO-PCA adversary $\mathcal{B}^A$ against KEM such that*

$$\begin{aligned} \text{Adv}_{\text{OCAKE}}^{\text{BPR}}(\mathcal{A}) &\leq \frac{n_a}{|\mathcal{D}|} + n_a^2 \cdot \eta_{\text{KGen}} + \frac{|\mathcal{D}|^2}{|\mathcal{K}_{pw}|} + \frac{80e^2 q_H^3 + 2}{|\mathcal{T}|} + n_a \cdot \delta_{\text{KEM}} + \\ &\quad + 4 \cdot \\ &\quad + 2 \cdot \left(\frac{1}{(1 - \delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{ANO-PCA}} + \varepsilon_{\text{SIM}}(\mathcal{B}^A) \right) \\ &\quad + 2 \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{IND-CPA}}(\mathcal{B}^I) + 2 \cdot \frac{q_H}{|\mathcal{K}|} \end{aligned}$$

and the running time of $\mathcal{B}^I$, and $\mathcal{B}^A$ is about that of $\mathcal{A}$.

To obtain this bound, we update the classical BPR proof for OCAKE [AHH⁺23] to capturing quantum attackers. We give an overview over the classical proof's game-hops in Table Fig. 6, indicate which game-hops we need to update, the new proof techniques we use to do so, and the resulting loss.

To summarize our updates, firstly, we lift all game-hops that use random oracle properties (games G_5, G_{13}) to the QROM. Secondly, for a concise presentation of how to apply our simulator properties, we fold all game-hops that use IC properties (games G_3, G_4, G_6, G_{10}) into a single game (G_{10}). The resulting upgraded proof is fully adapted to capturing quantum attackers.

We describe each game-hop in this section and give the intuition that underlies them. For the sake of completeness, we provide detailed reductions (including pseudocode) in Appendix B. For the reader's convenience, we also recall the details of the BPR security model in Appendix B.

We will use the notational convention $\text{Adv}_i := |\Pr[G_i(\mathcal{A}) \Rightarrow 1]|$.

Game	Description	Update?	Loss
G ₀	Original BPR game		
G ₁	Abort on KGen Collision.	Not necessary	$n_a^2 \cdot \eta_{\text{KGen}}$
G ₂	Abort on KDF Collision.	Not necessary	$\frac{ \mathcal{D} ^2}{ \mathcal{K}_{pw} }$
G ₃	IC lazy sampling w/ abort	This work (SIM)	-
G ₄	Prevent IC collisions	This work (SIM)	-
G ₅	Abort on Tag Collision.	This work (CO)	$\frac{80e^2 q_H^3 + 2}{ \mathcal{T} }$
G ₆	Sample IC using KGen	This work (SIM)	-
G ₇	Do not decrypt honest c .	Not necessary	$n_a \cdot \delta_{\text{KEM}}$
G ₈	Abort on tag under correct pw.	This work (eCO)	$\Pr[\text{corrPW}]$
G ₉	Make eCO extractions online.	This work (eCO)	$n_a \cdot (n_s + q_H) \sqrt{2/ \mathcal{T} }$
G ₁₀	Randomize public key.	This work (SIM)	$\frac{1}{(1-\delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{ANO-PCA}}(\mathcal{B}^A) + \varepsilon_{\text{SIM}}(\mathcal{B}^A)$
G ₁₁	Randomize pre-key.	Not necessary	$\text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{IND-CPA}}(\mathcal{B}^I)$
G ₁₂	Move eCO extractions to end.	This work (eCO)	$n_a \cdot (n_s + q_H) \sqrt{2/ \mathcal{T} }$
G ₁₃	Randomize Tag, Session Key.	This work (CO)	$\frac{q_H}{ \mathcal{K} }$
G ₁₅	Randomize Passwords.	Not necessary	0

Fig. 6. Game-hops in the classical BPR proof for OCAKE. The game-hops that needed updating use new techniques such as compressed oracles (CO), extractable compressed oracles (eCO), and the simulator defined in this work (SIM).

Game G₀: Original BPR game. The BPR oracles are exactly as in the existing BPR proof (see App. B). H, KDF and IC are quantum-accessible.

Game G₁: Handling collisions of KEM key pairs. (Unchanged) The game aborts whenever there are at least two sessions that sampled the same ephemeral KEM key pair (pk, sk) . Let η_{KGen} be the collision probability of KGen. Since games G₀ and G₁ are identical unless a collision occurs, we have $|\text{Adv}_1 - \text{Adv}_0| = \Pr[\text{KDFColl}] \leq n_a^2 \cdot \eta_{\text{KGen}}$.

Game G₂: Abort on KDF Collision. The game aborts whenever two passwords lead to a collision in the key derivation function that maps passwords to block cipher keys. This bound only depends on the choice of KDF and dictionary $\mathcal{D}$, not on any adversarial input. The resulting bound thus is statistical. Concretely, if KDF is a uniformly random function $f : \mathcal{D} \rightarrow \mathcal{K}_{pw}$, the probability that two inputs pw, pw' are mapped to the same output k_{pw} can be upper-bounded by the birthday collision bound: $|\text{Adv}_2 - \text{Adv}_1| \leq \Pr[\text{KDFColl}] \leq \frac{|\mathcal{D}|^2}{|\mathcal{K}_{pw}|}$.

Games G₃ and G₄: we fold them into game G₁₀.

Game G₅: Abort on Tag Collision. One of the informal security goals of PAKE proofs is to rule out attacks where an adversary can *test* more than one password in a single online attack. One example of such an attack is the

exploitation of a tag collision: assume there exist two passwords pw, pw' s.t. the responder tag tag computed from them for a given transcript is the same. Then the attacker could win the game by submitting tag without having uniquely identified the right password.

Therefore, G_5 aborts whenever the attacker has found such a collision in H . To argue about the distance between G_5 and G_4 , we use the compressed oracle technique [CFHL21].

Let $\mathcal{A}$ be the adversary in the PAKE game. We define an adversary $\mathcal{B}$ in the collision-finding game that runs $\mathcal{A}$ and is successful if it outputs x, x' s.t. $H(x) = H(x')$ at the end of the game.

$\mathcal{B}$ simulates $\mathcal{A}$'s PAKE and random oracle queries. First, $\mathcal{B}$ replaces the random oracle with a compressed random oracle. The compressed random oracle allows $\mathcal{B}$ to record $\mathcal{A}$'s queries in a database.

Then, at the end of the game, $\mathcal{B}$ measures the database for collisions. We denote the partitioning of the output of the hash function into tag and session key as $H(x) = tag || SK =: H(x)[0] || H(x)[1]$. Then we can define as R the relation that describes a collision in the tag: for distinct x, x' and arbitrary sk, sk' , we have $H(x)[0] || sk = H(x')[0] || sk'$. Informally, this is equivalent to discarding the part that represents the session key, or projecting onto the space of the tag. Then we have $R \subseteq \mathcal{X}^l \times (\mathcal{T} \times \mathcal{SK})^l$ for $l = 2$.

$$CL := \{D | \exists x \neq x' : D(x) = D(x') \neq \perp\}$$

Then, by Corollary 4.2 [CFHL21, Corollary 4.2], we have that for success probability in the original game p and success probability in the game using the compressed random oracle p' , we have that $\sqrt{p} \leq \sqrt{p'} + \sqrt{\frac{2}{|\mathcal{T}|}}$.

We now argue about the success probability p' , closely following the reasoning in section 2.3 of [CFHL21], (*Finding a Collision (with Parallel Queries)*). Let $SZ_{\leq(s-1)}$ be a database of size at most $s - 1$. We set $k = 1$ as we do not model parallel queries, so we have using the transition probability of a database that does not contain a collision to one that *does*

$$\sqrt{p'} \leq \sum_{s=1}^q \mathbb{I}[SZ_{\leq(s-1)} \setminus CL \rightarrow CL] \leq \sum_{s=1}^q 2e \sqrt{10 \frac{q_H}{|\mathcal{T}|}} \leq 2qe \sqrt{10 \frac{q_H}{|\mathcal{T}|}}$$

where e is Euler's number. Therefore we conclude $p' \leq \frac{40q^3 e^2}{|\mathcal{T}|}$. From $\sqrt{p} \leq \sqrt{p'} + \sqrt{\frac{2}{|\mathcal{T}|}}$ we then see by squaring that $p \leq p' + \frac{2}{|\mathcal{T}|} + \sqrt{\frac{2p'}{|\mathcal{T}|}} \leq \frac{40e^2(q^3 + \sqrt{2}q^{3/2}) + 2}{|\mathcal{T}|} \leq \frac{80e^2 q^3 + 2}{|\mathcal{T}|}$. Finally, this allows us to bound the distance: $|\mathbf{Adv}_5 - \mathbf{Adv}_4| \leq p \leq \frac{80e^2 q^3 + 2}{|\mathcal{T}|}$.

Game G_6 is also folded into game G_{10} .

Game G_7 : Correctness. (Unchanged) In game G_7 , whenever there is a flow 2 query where the message was honestly generated by a matching session, we do not decapsulate to obtain the pre-key. Instead, if the message was generated by

a matching session, we use the pre-key that was encapsulated by that instance. Adversarially generated messages as well as ones that are forwarded from a non-matching session are decapsulated as before. As in the classical proof, the distance to the previous game can be bounded by $|\mathbf{Adv}_7 - \mathbf{Adv}_6| \leq n_a \cdot \delta_{\text{KEM}}$.

Game G₈: Abort on dishonest tags using correct pw. We now abort whenever the adversary actively interacts with a session, meaning the session is not executed between two honest parties, and submits a tag that was generated using the correct password. This models the case that the attacker successfully guessed the password and used it to attack an honest part, which can happen both for

1. initiators, in which case the attacker sent a responder tag tag that uses the correct password, and for
2. responders, in which case the attacker sent an apk that uses the correct password.

Here we focus on case 1, since the other case is handled by simulation SIM (see 3). We need to adapt previous classical reasoning about this change to capture that oracle $\mathbf{H}$ now is quantum-accessible – previous reasoning exploited that game and reductions can observe the issued queries to $\mathbf{H}$, thus being able to immediately notice that tag was generated with the correct password. As a quantum counterpart to this approach, we use the extractable QROM formalism [DFMS22]. This models quantum-accessible random oracles like $\mathbf{H}$ as an extractable compressed random oracle $\mathbf{eCO}$ that has two interfaces, random oracle interface $\mathbf{eCO.RD}$ and an extraction interface $\mathbf{eCO.E}_f$ that is defined relative to a function $f : X \times Y = T$, where X and Y are the random oracle’s domain and co-domain, respectively, and T is some other set. For our purposes, we identify X with the domain of $\mathbf{H}$, so $X := \mathcal{K} \times \mathcal{D} \times \mathcal{PK}^2 \times \mathcal{C}$, and $Y := \mathcal{T} \times \mathcal{SK}$, the tag and session key space. Since we want to isolate the first part of the output, we choose the function f defined by the projection onto the tag space $f(input, y = (tag, K)) = tag$. Extraction interface $\mathbf{eCO.E}_f$ takes as input a classical value $t \in T$. It does a quantum analogue to going through the random oracle queries and returning an x such that $f(x, \mathbf{H}(x)) = t$: it performs suitable measurements that collapse the oracle database, just enough so that the classical procedure would yield one particular outcome x for all parts of the superposition. For our choice of f , $\mathbf{eCO.E}_f(tag)$ simply returns a random oracle pre-image of $tag||sk$ for any sk . Equipped with this extraction interface, the game waits until the attacker has finished and then calls $\mathbf{eCO.E}_f$ on all ‘dishonest’ tries, i.e., all tags tag that were received by an honest initiator whose password was not corrupted, without tag having been computed during a preceding honest execution of the respective responder. The game aborts if there exists a dishonest tag such that $\mathbf{eCO.E}_f(tag)$ returns a pre-image (K, pw, apk, pk', c) such that (K, apk, pk', c) matches the values which the initiator computed during the session to which the tag belongs. Since we already ruled out random oracle collisions, this in particular means that pw is the correct password. We denote this event by corrPW . We note that the extraction queries themselves do not change $\mathcal{A}$ ’s view since they are performed only after

$\mathcal{A}$ finished [DFMS22, Th. 4.3, item 1].

We can now conclude that $|\mathbf{Adv}_8 - \mathbf{Adv}_7| \leq \Pr[\text{corrPW}]_{G_8}$. Like in previous (classical) proofs, the change of probability for corrPW can be traced through the subsequent games G_8 - G_{15} , by building anonymity and indistinguishability reductions that define their output based on corrPW .

Game G_9 : Notice correct pw guess already during game. In this game, we move the identification of corrPW into the game: instead of performing the extraction queries determining corrPW after running the attacker, we perform them already during the session runs. We perform them whenever the attacker sends a tag tag to an honest initiator whose password was not corrupted, without tag having been computed during a preceding honest execution of the respective responder. This change is in preparation for the following two game hops, where the reductions use corrPW to detect certain edge-cases of adversarial input that need to be addressed during the game.

To argue that this introduces little change, we use [DFMS22, Th. 4.3]: by properties 2.b and 2.c of Th. 4.3, any two subsequent queries to $\mathbf{eCO.R0}$ and $\mathbf{eCO.Ef}$ $8\sqrt{2\Gamma(f)/|\mathcal{K}| \cdot |\mathcal{T}|}$ -almost-commute, where $\Gamma(f) = \max_t \max_x |\{y \mid f(x, y) = t\}|$, which for our choice $f(x, (tag, K)) = tag$ equals $|\mathcal{K}|$. This means that we can commute the extraction queries into the game to right after when the respective Send query was received. We need to commute at most $n_a \cdot (n_s + q_H H)$ many times until the extractions are at the right place, thus $|\mathbf{Adv}_9 - \mathbf{Adv}_8| = |\Pr[\text{corrPW}_{G_9}] - \Pr[\text{corrPW}_{G_8}]| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}$.

Game G_{10} : Randomize public-key pk . In this game, we randomize the public key used to create the responder ciphertext. First we recall the setting: in order to unlink the first (apk) and second (c, tag) protocol message, we argue that the ciphertext c does not leak which ciphertext was used to create it. More formally, we replace the public key used to derive c with a fresh uniformly random one, and reduce the distance in the games induced by this change to the ANO-PCA security of KEM.

We give a reduction that simulates the PAKE game for an adversary $\mathcal{A}$ that distinguishes BPR game G_{10} from G_9 and uses $\mathcal{A}$ to solve its ANO-PCA challenge. Consider the different query patterns that the reduction has to address. First we note that sessions that are passively attacked (queries to **Execute**) or those where the initiator is honest can always be simulated without the use of programming in the ideal cipher: honest parties simply use the challenge public keys in place of their honest keys.

The more involved case is the one where the responder is honest, but the initiator is not: this setting involves an *adversarially chosen* encrypted public key apk that is decrypted by the honest responder into a public key. In order to move to game G_{10} , where ciphertexts c are independent of the real public keys (or more accurately, the real password), the reduction reprograms decrypted public keys to challenge public keys in the ANO-PCA game.

We now show this proof step using only reprogramming according to the capabilities of **SIM** described above.

In a first step, the reduction answers all queries to $\mathbf{E}$ and $\mathbf{D}$ using the respective oracles of $\mathbf{SIM}$. This change to $\mathbf{SIM}$ is not perfectly indistinguishable, therefore we have $\mathbf{Adv}_{10,1} \leq \varepsilon_{\mathbf{SIM}}$ where $\varepsilon_{\mathbf{SIM}}$ by definition of $\mathbf{SIM}$.

The challenge set $\mathcal{X}$ used to initiate $\mathbf{SIM}$ is equal to the first set of public keys $\{pk_{0,0}, pk_{0,1}, \dots, pk_{0,n_a}\}$ provided by the anonymity challenger. This is the set that indicates the ‘real’ public keys, while the set $\{pk_{1,0}, pk_{1,1}, \dots, pk_{1,n_a}\}$ indicates the independent keys used *after* the reduction. Now, whenever the adversary submits a value apk for a session associated with password pw and key $k \leftarrow \text{KDF}(pw)$, apk decrypts to a challenge public key whenever $\mathbf{SIM}$ programs successfully. For n_a actively attacked sessions, we have by definition of $\mathbf{SIM}$ that

$$\Pr[(a\vec{pk}, s\vec{id}) \leftarrow \mathcal{A}^{\mathbf{SIM}(\mathcal{X})} : \mathbf{D}_{\mathbf{SIM}}(k, apk_i) \in \mathcal{X} \forall (sid_i, apk_i) \in (s\vec{id}, a\vec{pk}_i)] = 1 - \delta_{\mathbf{SIM}}.$$

The reduction, on input apk , therefore decrypts apk to some challenge public key $pk_{0,j} \in \mathcal{X}$ and queries its challenger for a fresh anonymity challenge c related to that public key, and uses that in its response. That way, the adversary receives a ciphertext matching either pk_0 or pk_1 in each session, exactly as in the two games.

Finally, we note that since the reduction simulates all of $\mathcal{A}$ ’s oracles, so there is an edge case to consider: the case where $\mathcal{A}$ corrupts a party after it has queried the $\mathbf{Send}^0$ and $\mathbf{Send}^1$, but before it queries $\mathbf{Send}^2$ for matching sessions while forwarding messages. In this case, the oracle $\mathbf{Send}^2$ uses a challenge public key pk_0 that the attacker now *knows* due to the corruption. However, the reduction does not have the secret key associated with the public key, and it cannot decapsulate the ciphertext and recompute the tag. Therefore, when the oracle decides whether or not to reject an adversarially generated (c, tag) , the reduction has use the extraction interface of the extractable random oracle $\mathbf{eCO}$ to determine the pre-key that was used by the adversary. Then, the reduction can query its plaintext-checking oracle to decide if the message should be rejected or not. Since we switched to the extractable oracle in a previous game, this does not incur any additional loss here.

Finally, the reduction outputs 0 whenever $\mathcal{A}$ wins, and 1 else. Therefore, the reduction wins whenever $\mathcal{A}$ wins and $\mathbf{SIM}$ successfully programmed for all its queries and we have $\mathbf{Adv}_{10,2} \leq \frac{1}{(1-\delta_{\mathbf{SIM}})} \cdot \mathbf{Adv}_{\mathbf{KEM}}^{\mathbf{ANO-PCA}}$. We then arrive at the bound $|\Pr[\text{corrPW}_{G_{10}}] - \Pr[\text{corrPW}_{G_9}]| = \mathbf{Adv}_{10} \leq \mathbf{Adv}_{10,1} + \mathbf{Adv}_{10,2} \leq \frac{1}{(1-\delta_{\mathbf{SIM}})} \cdot \mathbf{Adv}_{\mathbf{KEM}}^{(n_s, n_a + 1) - \mathbf{ANO-PCA}} + \varepsilon_{\mathbf{SIM}}(\mathcal{B}^A)$.

Game G_{11} : Randomize pre-key K . (Unchanged) For all queries to the $\mathbf{Send}$ or $\mathbf{Execute}$ oracles where flag trivGuess is not raised before the query, we now randomize the pre-key K that is used to derive the final session key and the responder tag. For more details, see the pseudo-code in Figure 19. This change makes the pre-key independent of the ciphertext and the password for all fresh sessions. As previously, the change in success probability can be bounded using the indistinguishability advantage of $\mathbf{KEM}$: $|\mathbf{Adv}_{11} - \mathbf{Adv}_{10}| \leq \mathbf{Adv}_{\mathbf{KEM}}^{(n_s, n_a + 1) - \mathbf{IND-CPA}}(\mathcal{B}^I)$.

Game G_{12} : Move dishonest tag identification back to end of game.

In this game, we switch back to offline extraction, i.e., we only perform the extraction queries determining if a tag was valid after running the attacker. This is done in preparation for the next step that relies on measuring the compressed database at the end of the game. With the same reasoning as for game G_9 , we find $|\Pr[\text{corrPW}_{G_{12}}] - \Pr[\text{corrPW}_{G_{11}}]| = |\mathbf{Adv}_{12} - \mathbf{Adv}_{11}| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}$.

Game G_{13} : Randomize Tag & Session Key. In the penultimate step of the proof, we randomize the responder tag tag and the session key SK , i.e., we simply sample random values from $\mathcal{T}$ and $\mathcal{SK}$, and return them as tag and session key. Recall, that in Game G_{12} the tag and session key are computed as $(tag, SK) \leftarrow \mathcal{H}(K_{\S}; pw, apk, pk', c)$ where $K_{\S}$ is a fresh uniformly random sample from $\mathcal{K}$ that is not used anywhere else in the protocol. Hence, the values in both games follow the same marginal distribution. However, in Game G_{12} they are consistent with the random oracle, while in Game G_{13} they are not. Intuitively, an adversary can only notice this inconsistency in Game G_{13} if it ever learned the value of $\mathcal{H}$ at position $x^* = (K_{\S}; pw, apk, pk', c)$. Given that we simulate $\mathcal{H}$ using the compressed oracle technique, this means that the difference between this and the last game can be bounded by the probability that the final database D of the compressed oracle in Game G_{13} contains a value for x^* when measured in the end, i.e., $|\mathbf{Adv}_{13} - \mathbf{Adv}_{12}| \leq \Pr[D(x^*) \neq \perp]$. It remains to quantify $\Pr[D(x^*) \neq \perp]$. For this, note that in Game G_{13} , $K_{\S}$ is not used anywhere in the protocol anymore and therefore the whole view of the adversary is independent of $K_{\S}$. For that reason, we can delay the sampling of $K_{\S}$ until after we measured the database D . The database D has q_H entries. The probability that one of them is of the form $(K_{\S}; pw, apk, pk', c)$ for any value of $K_{\S}$ is therefore upper bounded by $1/|\mathcal{K}|$ and therefore $|\mathbf{Adv}_{13} - \mathbf{Adv}_{12}| \leq \Pr[D(x^*) \neq \perp] \leq \frac{q_H}{|\mathcal{K}|}$.

Game G_{14} : Randomize Session Key is folded into the previous game.

Game G_{15} : Randomize Passwords. It remains to upper-bound corrPW . In this game, the attacker's view is completely independent of the chosen passwords, up to corrupted ones. This means that we can replace all non-corrupted passwords with fresh ones at the end, after running the attacker, and defining event corrPW with respect to the resampled ones. We note that there exists exactly one pre-image (K, pw, apk, pk', c) per tag that could trigger corrPW , and that the simulation SIM does not raise this flag as it did in the previous proof.

We can now bound the probability of corrPW in game G_8 , using the number of send queries and the password distribution. Assuming a uniform distribution on a password dictionary of size $|\mathcal{D}|$, and upper-bounding $\mathcal{A}$'s number of send queries by n_a , $\Pr[\text{corrPW}_{G_{15}}] \leq \frac{n_a}{|\mathcal{D}|}$. And therefore $\Pr[\text{corrPW}_{G_8}] \leq \sum_{i=8}^{15} \Pr[\text{corrPW}_{G_i}]$. Since the passwords and session keys are completely random from the adversary's view, we have that $|\Pr[G_{15}(\mathcal{A}) \Rightarrow 1]| = \frac{1}{2}$. We can

finally sum up the terms and state the bound:

$$\begin{aligned}
\text{Adv}_{\text{OCAKE}}^{\text{BPR}}(\mathcal{A}) &\leq \sum_{i=0}^{15} |\Pr[G_i(\mathcal{A}) \Rightarrow 1]| \leq \frac{n_a}{|\mathcal{D}|} + n_a^2 \cdot \eta_{\text{KGen}} + \frac{|\mathcal{D}|^2}{|\mathcal{K}_{pw}|} + \frac{80e^2 q_H^3 + 2}{|\mathcal{T}|} \\
&\quad + n_a \cdot \delta_{\text{KEM}} + 4 \cdot n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|} + \\
&\quad 2 \cdot \left(\frac{1}{(1 - \delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{ANO-PCA}} + \varepsilon_{\text{SIM}}(\mathcal{B}^A) \right) \\
&\quad + 2 \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{IND-CPA}}(\mathcal{B}^I) + 2 \cdot \frac{q_H}{|\mathcal{K}|}
\end{aligned}$$

We give an intuition on the game hops in the proof of CAKE and the steps needed to lift the proof into the quantum setting in Section A.1.

5 Necessity of SIM for OCAKE

In this section we prove that any generic security proof for (O)CAKE in the BPR model necessitates the use of programming, with the programming as captured in our definition of a programming simulator SIM (Definition 11). We show that (O)CAKE is vulnerable if the used KEM does not satisfy anonymity. Consequently, any proof that does not want to require statistical anonymity must include a reduction from KEMs anonymity. We show via a meta-reduction that there exists a class of KEMs for which a reduction from anonymity either must program the IC in the above sense or it is already able to solve anonymity itself. Thus, any security proof that wants to cover these kinds of KEMs must contain a reduction that implements a programming SIM. As we show, this class of KEMs includes ones that are obtained from a PKE scheme via the full FO transform, assuming they have ANO-CPA and that their explicitly rejecting counterpart satisfies a weak robustness property called hROB. This class contains Kyber and likely most other relevant pq KEM – the only property that was not previously shown is hROB, which we show for FO-based KEMs at the end of this section.

To explain a challenge we are facing and how it limits the class of covered KEMs to the aforementioned, note that our meta-reduction has to provide the reduction with a successful adversary. Other meta-reduction do this by rewinding the reduction, or the use of strong idealized models like the algebraic group model. In contrast, we use (Q)ROM techniques to enable our adversary to solve anonymity of instances other than the target instance it wants the reduction to solve. More specifically, we show that for this KEM class, anonymity does not hold statistically as a ciphertext produced under one of two public keys is unlikely to be valid under the other public key. Hence we can break anonymity given an oracle that decides validity of a ciphertext under a given public key. In addition, we have to show that access to such a Valid oracle does not make the task of solving anonymity for a given pair easier of Valid is muted on this target. We do so, by showing that a (meta-)reduction can simulate the Valid oracle for FO-transformed KEMs consistently towards an adversary except on

the actual anonymity challenge. The beauty is that as long as the reduction does not implement **SIM**, we are guaranteed that the public keys for which our meta-reduction has to implement **Valid** are all different from the challenge (and otherwise, we have a reduction that implements **SIM**).

Anonymity is Necessary. In proofs of PAKE, it is important to rule out dictionary attacks that would allow an attacker to use offline computations to determine the password used in some transcripts. This is why there is often a step that un-links the protocol messages from each other and the password. An example is the case of anonymity for OCAKE: the first message is an encryption of an ephemeral public key under the password. If an adversary were able to learn too much about this public key from the second protocol message (here an encapsulation under this public key), this would lead to a dictionary attack that tries to decrypt the first message and checks if the resulting public key agrees with the learned information.

To formalize this intuition, we show how to translate an attack on anonymity of KEM into an attack against the security of OCAKE. Let $\mathcal{B}$ be an algorithm in the anonymity game against KEM with advantage $\text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}(\mathcal{B})$. Recall that by definition, whenever $\mathcal{B}$ receives some ciphertext c , $\mathcal{B}$ can determine which of two public keys pk_0, pk_1 was used to create that ciphertext with advantage

$$\text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}(\mathcal{B}) = |\Pr[\text{ANO-PCA}^0(\mathcal{B}) \Rightarrow 0] - \Pr[\text{ANO-PCA}^1(\mathcal{B}) \Rightarrow 0]|$$

i.e., $\mathcal{B}$ outputs 0 whenever it guesses pk_0 was used, and 1 if pk_1 . We argue that $\mathcal{B}$ can be used to launch an attack on the protocol. We describe the adversary $\mathcal{A}$ against the security of OCAKE in figure 7.

First, $\mathcal{A}$ obtains a transcript apk, c, tag . $\mathcal{A}$ then chooses two passwords pw, pw' from the dictionary and queries $D(pw, apk)$ and $D(pw', apk)$ to obtain candidate public keys pk, pk' . For dictionary $\mathcal{D}$ and two randomly selected passwords, the probability that one of the two passwords was the correct one for that session is $\frac{2}{|\mathcal{D}|}$ ⁴. Next, $\mathcal{A}$ runs $\mathcal{B}$ on challenge values c, pk, pk' to receive a bit b , and selects the more likely password based on this.

Finally, $\mathcal{A}$ finishes the attack by running the sender side of the protocol using pw^* and sending the output to the receiver. The probability of $\mathcal{A}$ using the correct password in it's attack is therefore increased by the anonymity advantage of $\mathcal{B}$, provided the correct password was among the two that were sampled. This attack can be improved by running the test on all pairs of passwords in the dictionary, ensuring that the real public key was queried, and taking a majority vote. However, this requires additional assumptions of $\mathcal{B}$'s internal workings.⁵ From this we conclude that the attack's success probability is increased by $\frac{2}{|\mathcal{D}|} \cdot \text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}$. Given that $|\mathcal{D}|$ is cryptographically small by assumption,

⁴ In the BPR model passwords used are assumed to be from a uniform distribution.

⁵ We thank an anonymous reviewer for finding a bug in the previous algorithm that assumed $\mathcal{B}$'s output is random in case the ciphertext was tied to neither public key. While this assumption does not generally hold, it holds for the adversary we build as part of our meta-reduction later!

Algorithm Attack	
19	$P, P' \leftarrow \mathcal{P}^2$ // Choose parties to attack
20	$(apk, c^*, tag^*) \leftarrow \text{Execute}(P, P')$
21	$pw, pw' \leftarrow \mathcal{D}^2$ // Sample two passwords to distinguish
22	for pw, pw' : $pk[pw] \leftarrow \text{BC}^{-1}(pw, apk)$
23	$pk^* \leftarrow \mathcal{B}(c, pk[pw], pk[pw'])$ // Compute best guess
24	$pw^* \leftarrow pw[pk^*]$ // select appropriate password
25	$(pk^*, sk^*) \leftarrow \text{KGen}$ // Attack using candidate password
26	$apk^* \leftarrow \text{BC}(pw^*, pk^*)$
27	$(c^*, tag^*) \leftarrow \text{Send}^1(P', apk^*)$
28	$K^* \leftarrow \text{Decap}(sk^*, c^*)$
29	$SK \leftarrow \text{KDF}(\dots, K^*)$
30	$SK^* \leftarrow \text{Test}(P')$
31	if $SK == SK^*$ return 0
32	else return 1

Fig. 7. Attack Algorithm for adversary $\mathcal{A}$ using anonymity adversary $\mathcal{B}$ to attack security of OCAKE. The attack first obtains a ciphertext, computes the the most likely password based on the anonymity adversary, and then actively attacks a session with that password.

it follows that whenever KEM is not anonymous, OCAKE is vulnerable to this attack.

Any successful anonymity reduction must program. We now use the meta-reduction technique to investigate how any successful reduction between the security of OCAKE and the anonymity of KEM must program the implementation of the block cipher (or its modeling). More precisely, we prove Theorem 2 which says that there can be no reduction from the anonymity property of KEM to some game hop in the proof *unless* the reduction implements a programming simulator SIM that is able to program the block cipher as defined in Definition 11. To that end, we show two complementary cases separated on a flag that indicates if the ‘programming’ condition is met: any reduction that does not cause the flag is itself a successful anonymity adversary (Lemma 15) whereas any reduction that does has programmed exactly according to the definition (Lemma 16). We proceed as follows. As explained in the beginning of this section, this proof gives a meta-reduction that works for a certain class of KEM. We call this class **Valid-resistant Anonymous KEM** and give a definition in Definition 14. Intuitively, this class of KEM achieves anonymity even when the adversary has access to a validity oracle that given a public key pk and a ciphertext c decides if c is a valid ciphertext under pk (rejecting if c or pk are part of the anonymity challenge). We show that given a **Valid** oracle one can decide anonymity for δ -correct KEM that are **hROB** in their explicitly rejecting version (Lemma 18). Using this we show that the meta-reduction can implement an adversary against OCAKE combining the **Valid** attack on anonymity with the above anonymity attack against OCAKE.

Then, we show that KEM obtained via the full FO transform (including de-randomization in what is commonly referred to as the T-transform [HHK17]) are **Valid-resistant Anonymous KEM** (Lemma 19), i.e., the **Valid** oracle does not

```

Oracle  $\text{Valid}^*(c, pk)$ 
01 if  $c = c^*$  return  $\perp$ 
02  $sk \leftarrow pk.\text{compute\_secret\_key}$  //unbounded algorithm
03  $k \leftarrow \text{Decap}^*(sk, c)$ 
04 if  $k \neq \perp$  return True
05 else return False

```

Fig. 8. Validity oracle **Valid** outputs True if and only if c decrypts successfully under some secret key associated with public key pk . Decap^* is an algorithm that runs decapsulation, but indicates decryption failures even for implicitly rejecting schemes. This can be instantiated in various ways, e.g. using a re-encryption check.

help in solving an anonymity challenge (under the constraint that it is muted on elements of that challenge). Afterwards, we assemble the attack algorithm outlined above and the two lemmas to prove Lemmas 15 and 16. We now start with the definition of **Valid-resistant Anonymous KEM**. In this definition, we relate the advantage of two adversaries against ANO-PCA (c.f., Fig. 2) where one gets access to the additional validation oracle mentioned above. This oracle is described in Fig. 8. In the definition of this oracle, we assume an unbounded challenger that can brute-force secret keys to find a valid secret key for a given public key. We denote this brute-force algorithm as `compute_secret_key`. We will later show that for KEM obtained via the FO-transform, one can efficiently simulate this oracle via the random oracle that is used in the T-transform to de-randomize the initial scheme.

Definition 14 (Valid-resistant Anonymous KEM). *Let KEM be a key encapsulation mechanism with public-key space $\mathcal{PK}$ and key space $\mathcal{K}$. KEM is $\varepsilon_{\text{valid}}$ -Valid-resistant anonymous if the difference between the ANO-PCA-advantage of an adversary $\mathcal{B}$ with access to an additional oracle **Valid** as in Fig. 8 to that of a regular anonymity adversary $\mathcal{A}$ in the ANO-PCA game is bounded by $\varepsilon_{\text{valid}}$. I.e., we have that $\text{Adv}_{\text{ANO-PCA}}^{\text{KEM}}(\mathcal{B}^{\text{Valid}}) \leq \text{Adv}_{\text{ANO-PCA}}^{\text{KEM}}(\mathcal{A}) + \varepsilon_{\text{valid}}$.*

Theorem 2 states that there cannot exist a non-programming reduction. To prove this claim, we give a meta-reduction $\mathcal{M}$ and show that: whenever $\mathcal{R}$ does not program, it solves the anonymity challenge without the adversary and when it does, it does so according to the definition of SIM. This is formalized in Proposition 17. The meta-reduction is given in pseudocode in Fig. 9.

Lemma 15 ($\mathcal{M}$ is a successful anonymity adversary whenever $\mathcal{R}$ does not program). *For Valid-resistant anonymous, honest robust and δ_{KEM} -correct KEM, the meta-reduction in figure 9 wins the anonymity game whenever $\mathcal{R}$ does, provided that **program** does not occur:*

$$\Pr[\text{G}_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1] \geq \Pr[\text{G}_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\mathcal{M}}}) \Rightarrow 1 \mid \neg \text{program}]$$

Lemma 16 ($\mathcal{M}$ implements a programming simulator according to SIM whenever $\mathcal{R}$ programs). *For Valid-resistant anonymous, honest robust and*

δ_{KEM} -correct KEM, the meta-reduction in figure 9 succeeds in programming according to SIM for any simulation adversary $\mathcal{A}_{\text{SIM}}$ whenever `program` occurs:

$$\Pr[(apk, sid) \leftarrow \mathcal{A}_{\text{SIM}}^{\mathcal{M}(\mathcal{R}.\text{ESIM}, \mathcal{R}.\text{DSIM})} : D_{\text{SIM}}(K[sid], apk) \in \mathcal{X}] = \Pr[\text{program}]$$

Therefore the metareduction using the reduction to implement a programming simulator SIM has programming failure probability:

$$\delta_{\text{SIM}} = 1 - \Pr[\text{program}].$$

Proposition 17 (Any successful reduction either breaks anonymity or programs.). *Let $\mathcal{R}$ be a generic reduction from ANO-PCA security to the distinguishing problem in the proof of OCAKE with advantage $\text{Adv}_{\text{ANO-PCA}}^{\mathcal{R}}(\mathcal{A})$ given a distinguishing adversary $\mathcal{A}$. Then, for any choice of reduction $\mathcal{R}$, $\mathcal{M}$ succeed in either the anonymity or simulation game. More formally, the success probability $\mathcal{R}$ is upper-bounded by it's ability to program or solve anonymity:*

$$\Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\text{M}}}) \Rightarrow 1] \leq \Pr[G_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1] + \Pr[G_{\text{SIM}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1]$$

Proof (Proposition 17). We split the success probability of any reduction $\mathcal{R}$:

$$\begin{aligned} \Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\text{M}}}) \Rightarrow 1] &= \underbrace{\Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\text{M}}}) \Rightarrow 1 \mid \neg \text{program}]}_{\substack{\leq \Pr[G_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1] \\ (\text{Lemma 15})}} \cdot \underbrace{\Pr[\neg \text{program}]}_{\leq 1} \\ &\quad + \underbrace{\Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\text{M}}}) \Rightarrow 1 \mid \text{program}]}_{\leq 1} \cdot \underbrace{\Pr[\text{program}]}_{\substack{\leq (1 - \delta_{\text{SIM}}) \\ (\text{Lemma 16})}} \\ &\leq \Pr[G_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1] + (1 - \delta_{\text{SIM}}). \end{aligned}$$

□

Proof of Lemma 15. The success of the meta-reduction 9 in the anonymity game is shown in two parts: First we show that to the reduction, the meta-reduction behaves as a successful anonymity adversary. For this we show how to implement the anonymity attack from above using the `Valid` oracle. Second, we show that the meta-reduction does not trivialize the challenge for the reduction. To do so, we show that the internal oracles can be simulated when the KEM is obtained via the FO-transform, including the T-transform. From that it follows that there can be no general, non-programming reduction, not even a reduction that is non-generic but applies to FO-transformed schemes (with the additional properties laid out above).

Proof of Lemma 15 (part 1): The meta-reduction provides a successful adversary. By definition, a PAKE adversary wins its game whenever it distinguishes a session key from random. We show the meta-reduction $\mathcal{M}$'s attack algorithm instantiated using the `Valid` oracle behaves as a successful PAKE adversary leveraging non-anonymity of the scheme. We summarize the stages of the attack. The attacker:

Meta-reduction $\mathcal{M}^{\mathcal{R}}(\mathcal{A}_{\text{SIM}}, \mathcal{C}^{\text{ANO-PCA}})$

```

Simulated Adversary  $\mathcal{A}_{\mathcal{M}}$ 
01 forward  $\mathcal{A}_{\text{SIM}}$ 's queries to  $\mathcal{R}.\text{BC}, \mathcal{R}.\text{BC}^{-1}$ 
02  $(apk^*, sid) \leftarrow \mathcal{A}_{\text{SIM}}$  //Select point to attack
03  $(c^*, tag^*) \leftarrow \mathcal{R}.\text{Send}^1(P'[sid], apk^*)$ 
04 for  $pw \in \mathcal{D}$ :
05    $pk \leftarrow \mathcal{R}.\text{BC}^{-1}(pw, apk)$  // program if  $pk \in \mathcal{X}$  and  $pw = \text{PW}[P']$ 
06   if  $\text{Valid}(c, pk) : pw^* \leftarrow pw$  // unique and simulatable for certain schemes
07    $(pk^*, sk^*) \leftarrow \text{KGen}$  // Attack using candidate password
08    $apk \leftarrow \mathcal{R}.\text{BC}(pw^*, pk^*)$ 
09    $(c^*, tag^*) \leftarrow \mathcal{R}.\text{Send}^1(P'[sid], apk)$ 
10    $K^* \leftarrow \text{Decap}(sk^*, c^*)$ 
11    $SK \leftarrow \mathcal{R}.\text{KDF}(\dots, K^*)$ 
12    $SK^* \leftarrow \mathcal{R}.\text{Test}(P'[sid])$ 
13 if  $SK == SK^*$  return 0 to  $\mathcal{R}$ , else return 1
Meta-reduction  $\mathcal{M}^{\mathcal{R}}$ 
14  $b' \leftarrow \mathcal{R}^{\text{CANO-PCA}, \mathcal{A}_{\mathcal{M}}}$  //Run  $\mathcal{R}$  with access to challenger and adversary
15 output  $b'$ 

```

Fig. 9. Pseudocode for the meta-reduction algorithm using reduction $\mathcal{R}$ for challenges in the anonymity or simulation game. The meta-reduction is defined relative to the anonymity challenger $\mathcal{C}^{\text{ANO-PCA}}$, the simulation adversary $\mathcal{A}_{\text{SIM}}$, and the reduction $\mathcal{R}$. We have for the respective challenges of the anonymity and simulation that $\mathcal{X} = \{pk_0, pk_1\}$. The validity oracle can be efficiently simulated for **Valid**-resistant KEM (Lemma 20).

1. *queries* the reduction's on an authenticated public key apk to receive (c, tag) ,
2. *determines* the most likely public key of that session,
3. *derives* the password associated with that public key, and
4. *attacks* a fresh session using that password.

To determine the most likely public key, the meta-reduction makes use of the **Valid** oracle, querying on the ciphertext c received from the reduction and each of the candidate public keys pk_{pw_i} associated with the different passwords. We show that the attack succeeds whenever KEM is correct and its explicitly rejecting version is honest robust (hROB).

Lemma 18 (Attack using Valid oracle is successful if KEM is hROB and correct). *For KEM that is δ_{KEM} -correct and honest robust in its explicitly rejecting version, an attacker with access to a **Valid** oracle (Fig. 8) is a successful BPR PAKE adversary. More formally, we show that for an hROB adversary $\mathcal{B}$ and dictionary size $\mathcal{D}$:*

$$\Pr[\text{G}_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 1] \geq 1 - |\mathcal{D}| \cdot \text{Adv}_{\text{KEM}^\perp}^{\text{hROB}}(\mathcal{B}) - 2\delta_{\text{KEM}}.$$

Proof (Lemma 18). We prove Lemma 18 by analyzing the success probability of the meta-reduction's adversary. The game hops for this proof are shown in Fig. 10. The first change is semantic and $\Pr[\text{G}_1 \Rightarrow 1] = \Pr[\text{G}_0 \Rightarrow 1]$. We obtain G_2 from G_1 by aborting on correctness errors. For KEM that is δ_{KEM} -correct, we bound the distance to the previous game as $|\Pr[\text{G}_2 \Rightarrow 1] - \Pr[\text{G}_1 \Rightarrow 1]| \leq \delta_{\text{KEM}}$. G_3 adds another abort when decapsulation under the wrong pk does not result in a failure. The distance can be bounded via a hybrid argument (adding an

abort for each password at a time) and the honest robustness property of KEM: $|\Pr[G_3 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq |\mathcal{D}| \cdot \mathbf{Adv}_{\text{hROB}}^{\text{KEM}^\perp}$. Next, we inline the definition of the **Valid** oracle. This is a syntactic rewrite that does not change the success probability: $\Pr[G_4 \Rightarrow 1] = \Pr[G_3 \Rightarrow 1]$. Finally, we again abort on a correctness error in the second session and have $|\Pr[G_5 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \delta_{\text{KEM}}$. We see that $\mathcal{A}_{\mathcal{M}}^{\text{Valid}}$ always wins in game G_5 and get $\Pr[G_5 \Rightarrow 1] = 1$. In summary we get

$$\begin{aligned} \Pr[G_0] &\geq \underbrace{\Pr[G_5]}_1 - \underbrace{(\Pr[G_5] - \Pr[G_4])}_{\leq \delta_{\text{KEM}}} - \underbrace{(\Pr[G_4] - \Pr[G_3])}_0 - \underbrace{(\Pr[G_3] - \Pr[G_2])}_{\leq |\mathcal{D}| \cdot \mathbf{Adv}_{\text{hROB}}^{\text{KEM}^\perp}} \\ &\quad - \underbrace{(\Pr[G_2] - \Pr[G_1])}_{\leq \delta_{\text{KEM}}} - \underbrace{(\Pr[G_1] - \Pr[G_0])}_0 \\ &\geq 1 - |\mathcal{D}| \cdot \mathbf{Adv}_{\text{KEM}^\perp}^{\text{hROB}}(\mathcal{B}) - 2\delta_{\text{KEM}} \end{aligned}$$

□

We then have a lower bound for the success of any reduction $\mathcal{R}$ using this adversary:

$$\begin{aligned} \Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\mathcal{M}}^{\text{Valid}}}) \Rightarrow 1] &= \\ &\Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\mathcal{M}}^{\text{Valid}}}) \Rightarrow 1 | G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 1] \cdot \Pr[G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 1] \\ &\quad + \underbrace{\Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\mathcal{M}}^{\text{Valid}}}) \Rightarrow 1 | G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 0] \cdot \Pr[G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 0]}_{\geq 0} \\ &\geq \Pr[G_{\text{ANO-PCA}}(\mathcal{R}^{\mathcal{A}_{\mathcal{M}}^{\text{Valid}}}) \Rightarrow 1 | G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 1] \cdot \underbrace{\Pr[G_{\text{BPR}}(\mathcal{A}_{\mathcal{M}}^{\text{Valid}}) \Rightarrow 1]}_{\geq 1 - |\mathcal{D}| \cdot \mathbf{Adv}_{\text{KEM}^\perp}^{\text{hROB}}(\mathcal{B}) - 2\delta_{\text{KEM}}} \end{aligned}$$

We note here that none of the steps of the proof were dependent on the choice of apk or the previous queries to $\mathcal{R}.\text{IC}$, so the attack works for arbitrary choices of $\mathcal{A}_{\text{SIM}}$, including one that is that samples a random domain element instead to choose apk . This implies that $\mathcal{A}_{\text{SIM}}$ can also be simulated for this meta-reduction.

Proof of Lemma 15 (part 2): Anonymity is preserved under Validity oracle. To show that the validity oracle does not solve the challenge for the reduction, we investigate when ANO-PCA security is preserved under addition of the **Valid** oracle. More precisely, we show that for FO-transformed schemes any successful adversary that makes use of the **Valid** oracle can be converted into a general anonymity adversary in the QROM.

Lemma 19 (FO-transformed schemes are Valid-resistant Anonymous.).

*Let KEM be constructed using the T-transform [HHK17] from a PKE that is γ -spread and $\delta_{\text{corr}}^{\text{PKE}}$ -correct. Then access to an oracle **Valid** (Fig. 8) does not significantly improve the advantage of an adversary in the ANO-PCA game.*

*Concretely, for an ANO-PCA adversary $\mathcal{A}^{\text{Valid}}$ that additionally has access to the validity oracle **Valid**, making q_V many queries to **Valid**, q_P many queries to*

the plaintext checking oracle 1-PCO and q_G queries to random oracle G (the one used to derive randomness), we can construct an ANO-PCA adversary $\mathcal{B}$ without access to **Valid** s.t.

$$\begin{aligned} \text{Adv}_{\text{ANO-PCA}}^{\text{KEM}}(\mathcal{A}^{\text{Valid}}) &\leq \text{Adv}_{\text{ANO-PCA}}^{\text{KEM}}(\mathcal{B}) + q_V \cdot \delta_{\text{corr}}^{\text{PKE}} + q_V \cdot 2^{-\gamma} \\ &\quad + 12q_V(q_V + q_P + q_G) \cdot 2^{-\frac{\gamma}{2}}. \end{aligned}$$

Proof (Lemma 19). We show that ANO-PCA security implies security under the addition of the **Valid** oracle, using a reduction in the QROM. Adversary $\mathcal{B}$ runs $\mathcal{A}^{\text{Valid}}$ and simulates its oracles by forwarding the challenge public keys and the queries to the challenge and plaintext-checking oracles to its ANO-PCA challenger. When $\mathcal{A}^{\text{Valid}}$ outputs a bit, $\mathcal{B}$ forwards it to the ANO-PCA challenger and wins whenever $\mathcal{A}^{\text{Valid}}$ does.

Adversary $\mathcal{B}$ simulates the validity oracle $\text{Valid}_{\mathcal{B}}$ and random oracle $\mathbf{G}_{\mathcal{B}}$ as shown in Fig. 12. First we note that there are two cases where the simulated oracle behaves differently from the way is defined for an unbounded adversary:

- Case 1: $\text{Valid}_{\mathcal{B}}$ returns **False** while **Valid** does not. This case occurs whenever the adversary has found m s.t. $c = \text{PKE.Enc}(pk, m, \mathbf{G}(m))$ without querying $\mathbf{G}$ on m . Using Lemma 1 of [HHM22] and noticing that the simulated **Valid** oracle implements a decapsulation oracle with reject internally, we can bound the probability of this event as $q_V \cdot 2^{-\gamma}$ when PKE is γ -spread.
- Case 2: $\text{Valid}_{\mathcal{B}}$ returns **True** while **Valid** does not. This case corresponds to a correctness error of the KEM: $\mathcal{A}^{\text{Valid}}$ has found a ciphertext s.t. $\text{Decap}(sk, c) \neq K$ for $(c, K) \leftarrow \text{Encap}(pk)$. We can bound the probability of this event $q_V \cdot \delta_{\text{corr}}^{\text{PKE}}$.

Secondly, the simulated oracle extracts messages from the random oracle that match the ciphertext. To argue about this we again make use of the extractable QROM formalism [DFMS22] as well Lemma 1 of [HHM22].

The simulated $\text{Valid}_{\mathcal{B}}$ oracle uses the extraction interface eCO.E_f defined relative to a function $f : X \times Y = T$, where X and Y are the random oracle's domain and co-domain, respectively, and T is some other set. Here, we use the encryption function $\text{PKE.Enc}(pk, m, r)$ as our function f , i.e., we extract messages m s.t. for queried public key pk and ciphertext c , we have that $c = \text{PKE.Enc}(pk, m, \mathbf{G}(m))$. If the adversary has previously made a query that fulfills this predicate, the extraction returns this message m and the $\text{Valid}_{\mathcal{B}}$ oracle returns true. In the case that no such query was made, the extraction returns $\perp$ and the $\text{Valid}_{\mathcal{B}}$ oracle returns false.

Finally, we see using the technique from [HHM22] that the disturbance the online extraction incurs can be bounded as $8\sqrt{2\Gamma(f)/|\mathcal{R}|}$ with $\Gamma(f) = \max_t \max_x |\{y \mid R(x, y) = t\}| = 2^{-\gamma} \cdot |\mathcal{R}|$ since eCO.R0 and eCO.E_f almost-commute, and PKE is γ -spread.

We swap the eCO.R0 call that produces m with all calls to eCO.E that happen after the adversary submits c , including the calls inside the validity oracle and the plaintext-checking oracle. Therefore, we can bound the distance between the

games:

$$\begin{aligned} \mathbf{Adv}_{\text{KEM}}^{\text{ANO-PCA}}(\mathcal{A}^{\text{Valid}}) &\leq \mathbf{Adv}_{\text{ANO-PCA}}^{\text{KEM}}(\mathcal{B}) \\ &\quad + \underbrace{q_V \cdot \delta_{\text{corr}}^{\text{PKE}} + q_V \cdot 2^{-\gamma} + 12q_V(q_V + q_P + q_G) \cdot 2^{-\frac{\gamma}{2}}}_{=:\varepsilon_{FO}}. \end{aligned}$$

Therefore, we conclude that the KEM under consideration here are **Valid-resistant Anonymous** with ε_{FO} . $\square$

Honest robustness of FO-based KEMs. In this section, we used that the explicitly rejecting counterpart of FO-based KEMs has hROB. We now prove this property *if the KEM uses prefix hashing*, i.e., if it includes the public key into the hash when deriving the encryption randomness. In Section A, we show how to easily generalize this for FO-based KEMs *without* prefix hashing if the KEM satisfies an additional mild statistical property (weak collision freeness).

Lemma 20 (hROB of FO-based KEMs with prefix hashing). *Let PKE be a public-key encryption scheme that is γ -spread and for which two honestly generated public keys collide with probability at most η , and let KEM be the KEM obtained from PKE by applying the variant of the explicitly rejecting $\text{FO}^\perp$ transformation that computes the to-be-used encryption randomness as $r := \mathcal{G}(pk, m)$ instead of $r := \mathcal{G}(m)$. Then*

$$\mathbf{Adv}_{\text{KEM}}^{\text{hROB}}(\mathcal{A}) \leq 2^{-\gamma} + \eta.$$

Proof (Lemma 22). Plugging in the concrete algorithms for the FO-based KEM, $(c, \cdot) \leftarrow \$ \text{Encap}(pk_1)$ translates to $c \leftarrow \$ \text{Encrypt}(pk_1, m; \mathcal{G}(pk_1, m))$ for a uniformly chosen message m , and $k := \text{Decap}(sk_2, c)$ differs from $\perp$ if and only if c passes FO's re-encryption check under the public key that is included in sk_2 , that is, if $c = c'$, where $c' := \text{Encrypt}(pk_2, m'; \mathcal{G}(pk_2, m'))$ for $m' := \text{Decrypt}(sk_2, c)$.

If $m' \neq m$, k will differ from $\perp$ if and only if encrypting m' , a message different from m , accidentally hits c . Since we assume PKE to be γ -spread, for any key pair and any message m' , this happens with probability at most $2^{-\gamma}$. (The probability is taken over the randomness chosen in **Encrypt**, which is the output of random oracle $\mathcal{G}$ on a fresh message and consequently uniform.)

But even if $m' = m$, the two ciphertexts c and c' will still have used different encryption randomness if $pk_1 \neq pk_2$, in which case the bound $2^{-\gamma}$ still applies, and the case $pk_1 = pk_2$ can only happen with probability at most η . $\square$

Concluding the proof of Lemma 15. With these tools in hand, we can now conclude the proof of Lemma 15:

Proof (Lemma 15). Let KEM be **Valid-resistant anonymous**, **honest robust**, $\delta_{\text{KEM}}^{\text{correct}}$; let $\mathcal{R}$ be any reduction that reduces ANO-PCA of KEM to BPR security of OCAKE. Then the meta-reduction $\mathcal{M}^{\mathcal{R}}$ can use $\mathcal{R}$ to break ANO-PCA of KEM, unless $\mathcal{R}$ causes the **program** flag to be raised. Firstly, the adversary provided by the meta-reduction is successful according to lemma 18. Secondly,

from lemma 19 we conclude that for FO-transformed schemes, the adversary provided by the meta-reduction can be efficiently simulated, unless **program** occurs.

From these two points, we conclude that the meta-reduction $\mathcal{M}$ efficiently transforms the non-programming reduction $\mathcal{R}$ into a successful adversary in the anonymity game:

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) &= \Pr[\text{G}_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}})] \\ &= \Pr[\text{G}_{\text{ANO-PCA}}(\mathcal{M}^{\mathcal{R}}) \Rightarrow 1 \mid \neg \text{program}] \cdot \Pr[\neg \text{program}] \\ &= \Pr[\text{G}_{\text{ANO-PCA}}(\mathcal{R}) \Rightarrow 1] \cdot \Pr[\neg \text{program}] \end{aligned}$$

□

Proof of Lemma 16. We build a meta-reduction $\mathcal{M}$ to show that whenever the **program** flag is raised, the reduction $\mathcal{R}$ instantiates a programming simulator according to **SIM**. This meta-reduction is defined w.r.t. a simulation challenge $\mathcal{X}$, i.e., a set of public keys; and an adversary $\mathcal{A}_{\text{SIM}}$ that queries the oracles **E** and **D** to finally output a value sid, apk . The meta-reduction is said to program whenever at the end we have that for this apk and the session password $\text{pw}(\text{sid})$: $\text{D}(\text{pw}, \text{apk}) \in \mathcal{X}$ according to Definition 11.2. The meta-reduction also makes use of a challenger in the anonymity game since the reduction is only defined with respect to such a challenger, however this challenger can be simulated by the meta-reduction.

Proof (Lemma 16). Let **KEM** be **Valid-resistant** anonymous, honest robust, δ_{KEM} -correct; let $\mathcal{R}$ be any reduction that reduces **ANO-PCA** of **KEM** to **BPR** security of **OCAKE**. Then the meta-reduction $\mathcal{M}^{\mathcal{R}}$ using $\mathcal{R}$ instantiates **SIM** whenever $\mathcal{R}$ causes the **program** flag to be raised. I.e.,

$$\Pr[(\text{apk}, \text{sid}) \leftarrow \mathcal{A}_{\text{SIM}}^{\mathcal{M}(\mathcal{R}, \text{E}_{\text{SIM}}, \mathcal{R}, \text{D}_{\text{SIM}})} : \text{D}_{\text{SIM}}(K[\text{sid}], \text{apk}) \in \mathcal{X}] = \Pr[\text{program}].$$

We analyse the adversary: In the proof of Lemma 15 we showed that the adversary provided by the meta-reduction is successful according to Lemmas 18 and 19, unless $\mathcal{A}_{\mathcal{M}}$ receives a ciphertext c^* (in line 15) that was derived from one of the public keys in $\mathcal{X}$. This ciphertext c^* is either associated with the matching decrypted public key of the session, or an independent key. Therefore, we must have $\text{D}(\text{pw}, \text{apk}) = pk_0$ or $\text{D}(\text{pw}, \text{apk}) = pk_1$ for the password pw of the attacked party. Since we chose $\text{apk} = \text{apk}^*$ this implies $\text{D}(\text{pw}, \text{apk}^*) \in \mathcal{X}$, exactly as desired. Therefore we conclude that $\mathcal{M}$ implements **SIM** with failure probability $\delta_{\text{SIM}} = 1 - \Pr[\text{program}]$. □

```

Oracle  $\text{Valid}^{c^*, pk_0, pk_1}(c, pk)$ 
01 if  $c = c^*$  return False //reject mapped to False
02  $k \leftarrow \text{Decap}^*(sk[pk], c)$ 
03 if  $k \neq \perp$  return True
04 else return False
Game  $G_0, G_1$  Attack Algorithm //Inline reduction's send and test oracles
05  $pw^* \leftarrow \perp$  //Initialize guess
06 forward  $\mathcal{A}_{\text{SIM}}$ 's queries to  $\mathcal{R}.\text{BC}, \mathcal{R}.\text{BC}^{-1}$ 
07  $(apk^*, sid) \leftarrow \mathcal{A}_{\text{SIM}}$ 
08  $(c^*, tag^*) \leftarrow \mathcal{R}.\text{Send}^1(P'[sid], apk^*)$   $pk' \leftarrow \text{BC}^{-1}(pw[P'[sid]], apk^*)$ ;  $(c^*, K) \leftarrow \text{Encap}(pk')$ 
09 for  $(j, pw_j) \in (|D|, D)$ :
10    $pk_j \leftarrow \mathcal{R}.\text{BC}^{-1}(pw, apk^*)$  //program if  $\mathcal{R}.\text{BC}^{-1}(pw^*, apk^*) \in \mathcal{X}$ 
11   if  $\text{Valid}(c, pk_j) : pw^* \leftarrow pw_j$ 
12 fail if  $pw^* = \perp$  //Attack fails if there is no matching  $pk$ 
13  $(pk^*, sk^*) \leftarrow \text{KGen}$ 
14  $apk \leftarrow \mathcal{R}.\text{BC}(pw^*, pk^*)$ 
15  $(c^*, tag^*) \leftarrow \mathcal{R}.\text{Send}^1(P'[sid], apk)$   $pk' \leftarrow \text{BC}^{-1}(pw[P'[sid]], apk)$ ;  $(c^*, K) \leftarrow \text{Encap}(pk')$ 
16  $K' \leftarrow \text{Decap}(sk^*, c^*)$ 
17  $SK \leftarrow \mathcal{R}.\text{KDF}(\dots, K^*)$ 
18  $SK^* \leftarrow \mathcal{R}.\text{Test}(P'[sid])$   $SK_0 \leftarrow \mathcal{R}.\text{KDF}(\dots, K')$ ;  $SK_1 \leftarrow \$ SK$ ;  $SK^* \leftarrow SK_b$ 
19 if  $SK == SK^*$  return 0, else return 1
Hybrids  $G_2, G_3, G_4, G_5$  //Correctness, honest robustness, validity, correctness
20  $pw^* \leftarrow \perp$ 
21 forward  $\mathcal{A}_{\text{SIM}}$ 's queries to  $\mathcal{R}.\text{BC}, \mathcal{R}.\text{BC}^{-1}$ 
22  $(apk^*, sid) \leftarrow \mathcal{A}_{\text{SIM}}$ 
23 for  $(j, pw_j) \in (|D|, D)$ :  $pk_j \leftarrow \mathcal{R}.\text{BC}^{-1}(pw_j, apk^*)$ 
24  $pk' \leftarrow \text{BC}^{-1}(pw[P'[sid]], apk^*)$ 
25  $(c^*, K) \leftarrow \text{Encap}(pk')$  //If reduction randomizes key:  $(c^*, K) \leftarrow \text{Encap}(pk^{\$})$ 
26 win if  $\text{Decap}(sk[pk']) \neq K$  //Correctness ( $G_2$ )
27 for  $(j, pw_j) \in (|D|, D)$ : win if  $\text{Decap}^*(sk_j, c) \neq \perp$  and  $sk \neq sk[pk']$  //Honest robustness ( $G_3$ )
28 for  $(j, pw_j) \in (|D|, D)$ 
29   if  $\text{Valid}(c, pk_j) : pw^* \leftarrow pw_j$  if  $\text{Decap}^*(sk[pk_j], c) \neq \perp : pw^* \leftarrow pw_j$  //Validity ( $G_4$ )
30 fail if  $pw^* = \perp$  //fail occurs iff  $(c^*, K) \leftarrow \text{Encap}(pk^{\$})$ 
31  $(pk^*, sk^*) \leftarrow \text{KGen}$ 
32  $apk \leftarrow \mathcal{R}.\text{BC}(pw^*, pk^*)$ 
33  $pk'' \leftarrow \text{BC}^{-1}(pw[P'[sid]], apk)$  //  $pk'' \leftarrow pk^*$ 
34  $(c^*, K^*) \leftarrow \text{Encap}(pk'')$ ;  $K' \leftarrow \text{Decap}(sk^*, c^*)$   $K' \leftarrow K^*$  //Correctness ( $G_5$ )
35  $SK \leftarrow \mathcal{R}.\text{KDF}(\dots, K')$ 
36  $SK_0 \leftarrow \mathcal{R}.\text{KDF}(\dots, K')$ ;  $SK_1 \leftarrow \$ SK$ ;  $SK^* \leftarrow SK_b$ 
37 if  $SK == SK^*$  return 0, else return 1

```

Fig. 10. Hybrid games to show that the attack algorithm given by $\mathcal{M}$ is (1) successful and the public key used to compute c matches the correct password's public key and (2) fails whenever the public key was independent of the password provide that the scheme is correct and honest robust.

<u>Gen^ℓ</u> 01 $(pk, sk) \leftarrow \text{Gen}$ 02 $s \xleftarrow{\$} \mathcal{M}$ 03 $sk' := (sk, s)$ 04 return (pk, sk')	<u>Encaps(pk)</u> <u>Encaps_m(pk)</u> 09 $m \xleftarrow{\$} \mathcal{M}$ 10 $c := \text{Enc}(pk, m; G(m))$ 11 $K := H(m, c)$ <u>$K := H(m)$</u> 12 return (K, c)
<u>Decaps[⊥](sk, c)</u> <u>Decaps_m[⊥](sk, c)</u> 05 $m' := \text{Dec}(sk, c)$ 06 if $c \neq \text{Enc}(pk, m'; G(m'))$ or $m' = \perp$ 07 return $\perp$ 08 else return $K := H(m', c)$ <u>$K := H(m')$</u>	<u>Decaps^ℓ($sk' = (sk, s), c$)</u> <u>Decaps_m^ℓ($sk'(sk, s), c$)</u> 13 $m' := \text{Dec}(sk, c)$ 14 if $c \neq \text{Enc}(pk, m'; G(m'))$ or $m' = \perp$ 15 return $K := H(s, c)$ 16 else return $K := H(m', c)$ <u>$K := H(m')$</u>

Fig. 11. Algorithms of FO transformed KEM according to [HHK17].

<u>Reduction $\mathcal{B}(pk_0, pk_1, c)$</u> 38 $b' \leftarrow \mathcal{A}^{\text{Valid}_{\mathcal{B}}^{c^*}}(pk_0, pk_1, c)$ 39 return b'	<u>Simulated oracle $\text{Valid}_{\mathcal{B}}^{c^*, pk_0, pk_1}(pk, c)$</u> 04 if $c = c^*$ return $\perp$ 05 $m \leftarrow \text{eCO.E}_{c=\text{PKE.Enc}(pk, m)}(c, pk)$ 06 if $m \neq \perp$ return True 07 else return False
<u>Simulated oracle $G_{\mathcal{B}}(m)$</u> 01 $r \leftarrow G(m)$ 02 if $\text{PKE.Enc}(pk, m, r) = c^*$ return r 03 else return $\text{eCO.R0}(m)$	

Fig. 12. The simulated oracles of Adversary $\mathcal{B}$ reducing security under the additional validity oracle to ANO-PCA security.

References

- ACMT25. Gorjan Alagic, Joseph Carolan, Christian Majenz, and Saliha Tokat. The sponge is quantum indifferentiable. *Cryptology ePrint Archive*, Paper 2025/731, 2025.
- AHH⁺23. Nouri Alnahawi, Kathrin Hövelmanns, Andreas Hülsing, Silvia Ritsch, and Alexander Wiesmaier. Towards post-quantum secure pake-a tight security proof for oake in the bpr model. *Cryptology ePrint Archive*, 2023.
- AHHR24. Nouri Alnahawi, Kathrin Hövelmanns, Andreas Hülsing, and Silvia Ritsch. Towards post-quantum secure PAKE - A tight security proof for OAKE in the BPR model. In Markulf Kohlweiss, Roberto Di Pietro, and Alastair R. Beresford, editors, *CANS 2024: 23rd International Conference on Cryptology and Network Security, Part II*, volume 14906 of *Lecture Notes in Computer Science*, pages 191–212, Cambridge, UK, September 24–27, 2024. Springer, Singapore, Singapore.
- AHMW25. Nouri Alnahawi, David Haas, Erik Mauß, and Alexander Wiesmaier. SoK: PQC PAKEs - cryptographic primitives, design and security. *Cryptology ePrint Archive*, Report 2025/119, 2025.
- BCP⁺23. Hugo Beguinet, Céline Chevalier, David Pointcheval, Thomas Ricosset, and Mélissa Rossi. GeT a CAKE: Generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 2023: 21st International Conference on Applied Cryptography and Network Security, Part II*, volume 13906 of *Lecture Notes in Computer Science*, pages 516–538, Kyoto, Japan, June 19–22, 2023. Springer, Cham, Switzerland.
- BDF⁺11. Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 41–69, Seoul, South Korea, December 4–8, 2011. Springer Berlin Heidelberg, Germany.
- Bla06. John Black. The ideal-cipher model, revisited: An uninstantiable blockcipher-based hash function. In Matthew J. B. Robshaw, editor, *Fast Software Encryption – FSE 2006*, volume 4047 of *Lecture Notes in Computer Science*, pages 328–340, Graz, Austria, March 15–17, 2006. Springer Berlin Heidelberg, Germany.
- BPR00. Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 139–155, Bruges, Belgium, May 14–18, 2000. Springer Berlin Heidelberg, Germany.
- BV98. Dan Boneh and Ramarathnam Venkatesan. Breaking RSA may not be equivalent to factoring. In Kaisa Nyberg, editor, *Advances in Cryptology – EUROCRYPT’98*, volume 1403 of *Lecture Notes in Computer Science*, pages 59–71, Espoo, Finland, May 31 – June 4, 1998. Springer Berlin Heidelberg, Germany.
- CFHL21. Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of

- Lecture Notes in Computer Science*, pages 598–629, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- CHL⁺25. Alexandru Cojocaru, Minki Hhan, Qipeng Liu, Takashi Yamakawa, and Aaram Yun. Quantum lifting for invertible permutations and ideal ciphers. *Cryptology ePrint Archive*, Paper 2025/738, 2025.
- CMSZ19. Jan Czajkowski, Christian Majenz, Christian Schaffner, and Sebastian Zur. Quantum lazy sampling and game-playing proofs for quantum indistinguishability. *Cryptology ePrint Archive*, Paper 2019/428, 2019.
- CP25. Joseph Carolan and Alexander Poremba. Quantum one-wayness of the single-round sponge with invertible permutations, 2025.
- DAL⁺17. Jintai Ding, Saed Alsayigh, Jean Lancrenon, Saraswathy Rv, and Michael Snook. Provably secure password authenticated key exchange based on rlwe for the post-quantum world. In *Cryptographers’ Track at the RSA conference*, pages 183–204. Springer, 2017.
- DFMS22. Jelle Don, Serge Fehr, Christian Majenz, and Christian Schaffner. Online-extractability in the quantum random-oracle model. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 677–706, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- DKBY24. Vivek Dabra, Saru Kumari, Anju Bala, and Sonam Yadav. Sl3pake: Simple lattice-based three-party password authenticated key exchange for post-quantum world. *Journal of Information Security and Applications*, 84:103826, 2024.
- FF13. Marc Fischlin and Nils Fleischhacker. Limitations of the meta-reduction technique: The case of Schnorr signatures. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 444–460, Athens, Greece, May 26–30, 2013. Springer Berlin Heidelberg, Germany.
- FLR⁺10. Marc Fischlin, Anja Lehmann, Thomas Ristenpart, Thomas Shrimpton, Martijn Stam, and Stefano Tessaro. Random oracles with(out) programmability. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 303–320, Singapore, December 5–9, 2010. Springer Berlin Heidelberg, Germany.
- FO99. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology – CRYPTO’99*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554, Santa Barbara, CA, USA, August 15–19, 1999. Springer Berlin Heidelberg, Germany.
- FO13. Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.
- GMP22. Paul Grubbs, Varun Maram, and Kenneth G. Paterson. Anonymous, robust post-quantum public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 402–432, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.

- HHK17. Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017: 15th Theory of Cryptography Conference, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371, Baltimore, MD, USA, November 12–15, 2017. Springer, Cham, Switzerland.
- HHM22. Kathrin Hövelmanns, Andreas Hülsing, and Christian Majenz. Failing gracefully: Decryption failures and the Fujisaki-Okamoto transform. In Shweta Agrawal and Dongdai Lin, editors, *Advances in Cryptology – ASIACRYPT 2022, Part IV*, volume 13794 of *Lecture Notes in Computer Science*, pages 414–443, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland.
- HM24. Kathrin Hövelmanns and Christian Majenz. A note on failing gracefully: Completing the picture for explicitly rejecting Fujisaki-Okamoto transforms using worst-case correctness. In Markku-Juhani Saariinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 245–265, Oxford, UK, June 12–14, 2024. Springer, Cham, Switzerland.
- Hos25. Akinori Hosoyamada. Post-quantum security of keyed sponge-based constructions through a modular approach. Cryptology ePrint Archive, Paper 2025/1059, 2025.
- LLH24. You Lyu, Shengli Liu, and Shuai Han. Universal composable password authenticated key exchange for the post-quantum world. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 120–150. Springer, 2024.
- MMW24. Christian Majenz, Giulio Malavolta, and Michael Walter. Permutation superposition oracles for quantum query lower bounds. Cryptology ePrint Archive, Paper 2024/1140, 2024.
- MX23. Varun Maram and Keita Xagawa. Post-quantum anonymity of Kyber. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13940 of *Lecture Notes in Computer Science*, pages 3–35, Atlanta, GA, USA, May 7–10, 2023. Springer, Cham, Switzerland.
- PZ23. Jiaxin Pan and Runzhi Zeng. A generic construction of tightly secure password-based authenticated key exchange. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part VIII*, volume 14445 of *Lecture Notes in Computer Science*, pages 143–175, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- SA23. Kübra Seyhan and Sedat Akleylek. A new password-authenticated module learning with rounding-based key exchange protocol: Saber. pake. *The Journal of Supercomputing*, 79(16):17859–17896, 2023.
- SH19. Vladimir Soukharev and Basil Hess. Pqdh: a quantum-safe replacement for diffie-hellman based on sidh. *Cryptology ePrint Archive*, 2019.
- TY19. Shintaro Terada and Kazuki Yoneyama. Password-based authenticated key exchange from standard isogeny assumptions. In *Provable Security: 13th International Conference, ProvSec 2019, Cairns, QLD, Australia, October 1–4, 2019, Proceedings 13*, pages 41–56. Springer, 2019.
- Unr21. Dominique Unruh. Compressed permutation oracles (and the collision-resistance of sponge/SHA3). Cryptology ePrint Archive, Paper 2021/062, 2021.
- Unr23a. Dominique Unruh. Towards compressed permutation oracles. Cryptology ePrint Archive, Paper 2023/770, 2023.

- Unr23b. Dominique Unruh. Towards compressed permutation oracles. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023, Part IV*, volume 14441 of *Lecture Notes in Computer Science*, pages 369–400, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- Xag22. Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 551–581, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
- Zha19. Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part II*, volume 11693 of *Lecture Notes in Computer Science*, pages 239–268, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.

A Honest robustness of FO-KEMs without prefix hashing

In Section 5, we showed our simulator necessity result for an important class of KEMs, KEMs obtained via the full FO-transform. For this result, we needed that the explicitly rejecting counterpart of these KEMs has hROB. We showed this property (see Lemma 20) for such KEMs *if they use prefix hashing*, i.e., if they include the public key into their hashing (at least when deriving the encryption randomness). To broaden the result, we now argue hROB of FO-based KEMs *without* prefix hashing – with Lemma 22 below, we relate hROB of any such FO-based KEM to properties of the KEM’s underlying PKE scheme, γ -spreadness and a property we call weak collision freeness. Weak collision freeness captures that ciphertexts generated under one key pair will not likely decrypt to their originating plaintext under another key pair, where all values were sampled honestly. We first formalize weak collision freeness with Definition 21 and then give Lemma 22.

Definition 21 (Weak collision freeness). *Let PKE be a public-key encryption scheme. We say that PKE satisfies ρ -weak collision freeness iff*

$$\Pr \left[m' = m \left| \begin{array}{l} (pk_1, \cdot), (\cdot, sk_2) \leftarrow \$ \text{KGen}() \\ m \leftarrow \$ \mathcal{M} \\ c \leftarrow \$ \text{Encrypt}(pk_1, m) \\ m' := \text{Decrypt}(sk_2, c) \end{array} \right. \right] \leq \rho .$$

After proving Lemma 22, it only remains to analyze the required PKE properties.

Lemma 22 (hROB of FO-based KEMs). *Explicitly rejecting FO-based KEMs satisfy hROB if their underlying encryption scheme is sufficiently spread and weak collision free.*

Concretely, let PKE be a public-key encryption scheme that is γ -spread and ρ -weak collision free and for which two honestly generated public keys collide with probability at most η , and let $\text{KEM} := \text{FO}^\perp[\text{PKE}, \mathcal{G}, \mathcal{H}]$. Then

$$\text{Adv}_{\text{KEM}}^{\text{hROB}}(\mathcal{A}) \leq 2^{-\gamma} + \rho .$$

Proof (Lemma 22). We want to show that $2^{-\gamma} + \rho$ is an upper bound for

$$\text{Adv}_{\text{KEM}}^{\text{hROB}}(\mathcal{A}) = \Pr \left[k \neq \perp \left| \begin{array}{l} (pk_1, \cdot), (\cdot, sk_2) \leftarrow \$ \text{KGen}() \\ (c, \cdot) \leftarrow \$ \text{Encap}(pk_1) \\ k := \text{Decap}(sk_2, c) \end{array} \right. \right] .$$

The first part of the proof proceeds like the one of Lemma 20. Plugging in the concrete algorithms for the FO-based KEM, $(c, \cdot) \leftarrow \$ \text{Encap}(pk_1)$ translates to $c \leftarrow \$ \text{Encrypt}(pk_1, m; \mathcal{G}(m))$ for a uniformly chosen message m , and $k := \text{Decap}(sk_2, c)$ will differ from $\perp$ if and only if c passes FO’s re-encryption check, that is, if $c = \text{Encrypt}(pk_2, m'; \mathcal{G}(m'))$ for $m' := \text{Decrypt}(sk_2, c)$.

We make a case distinction: either $m' = m$ or $m' \neq m$. If $m' \neq m$, k will differ from $\perp$ if and only if encrypting m' , a message different from m , accidentally hits c . Since we assume PKE to be γ -spread, for any key pair and any message m' , this happens with probability at most $2^{-\gamma}$. This probability is taken over the randomness chosen in **Encrypt** which here is the output of random oracle $\mathbf{G}$ on a fresh message and consequently uniform.

The other case, $m' = m$, is exactly the case that encrypting a random message m under one key pair and then decrypting the result under an independently sampled one still yields the original message. By assumption, we can bound the probability of this case occurring by ρ . (Note that in our analysis, we can replace the used encryption randomness $\mathbf{G}(m)$ by uniform randomness since $\mathbf{G}$ is a random oracle and we are interested in a statistical property.) $\square$

CAKE

We give the description of the CAKE protocol in figure 13.

Initiator I (Client)	Protocol CAKE	Responder R (Server)
Password pw		Password pw
Public Key		
$(pk, sk) \leftarrow \$ \text{KGen}$		
$e_1 \leftarrow \text{BC}_1(pw, pk)$	$\xrightarrow{e_1}$	$pk' \leftarrow \text{BC}_1^{-1}(pw, e_1)$
Session Pre-Key		
$c' \leftarrow \text{BC}_2^{-1}(pw, e_2)$	$\xleftarrow{e_2}$	$(c, K) \leftarrow \$ \text{Encap}(pk')$
$K' \leftarrow \text{Decap}(sk, c')$		$e_2 \leftarrow \text{BC}_2(pw, c)$
Key Derivation		
$SK' \leftarrow \mathbf{H}(pk, c', K', pw)$		$SK \leftarrow \mathbf{H}(pk', c, K, pw)$
output SK'		output SK

Fig. 13. The CAKE protocol, using a key encapsulation mechanism $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$, hash function $\mathbf{H}$ and a block cipher $\text{BC} = (\text{BC}, \text{BC}^{-1})$. In both cases, existing proofs model the block cipher as an ideal cipher.

A.1 Security of CAKE under SIM

Security of the CAKE protocol has been shown in the ROM and ICM in the universal composability framework [BCP⁺23] and in the BPR model [PZ23]. In the proof of [PZ23], there are four game hops that make use of programmability.⁶ Additionally, in game G_7 it is argued that reductions make arguments

⁶ Appendix 14 gives a more thorough analysis of the necessary changes and the role of the IC in the proof.

Game	Description	Update?	Loss
G_{-1}	Original BPR game		
G_0	Collision Events.	No change.	$S^2(\eta_{pk} + \eta_{ct}) + \frac{(q_1^2 + S^2)}{ \mathcal{E}_1 } + \frac{(q_2^2 + S^2)}{ \mathcal{E}_2 } + \frac{q_1^2}{ \mathcal{PK} } + \frac{q_2^2}{ \mathcal{C} } + \frac{q_H^2 + S^2}{ \mathcal{SK} }$
G_1	Freshness.	No change.	0
G_2	Sample D_1 using KGen.	Use SIM.	-
G_3	Randomize Session keys (passive).	No change.	$\text{Adv}_{\text{KEM}}^{(S,1)\text{-OW-PCA}}(\mathcal{B}_2)$
G_4	Randomize ciphertexts (passive).	No change.	$\text{Adv}_{\text{KEM}}^{(S,1)\text{-ANO}}(\mathcal{B}_3)$
G_5	Randomize protocol messages (passive).	No change.	0
G_6	Do not decapsulate honest.	No change.	$S \cdot \delta_{\text{KEM}}$
G_7	Extract password.	May require extraction.	open
G_8	Randomize pre-key.	Use SIM.	$(1 - \delta_{\text{SIM}}) \cdot \text{OW-rPCA} + \varepsilon_{\text{SIM}}$
G_9	Randomize pre-key.	Use SIM.	$(1 - \delta_{\text{SIM}}) \cdot \text{OW-PCA} + \varepsilon_{\text{SIM}}$
G_{10}	Randomize public keys.	Use SIM.	$(1 - \delta_{\text{SIM}}) \cdot \text{ANO-PCA} + \varepsilon_{\text{SIM}}$
G_{11}	Randomize protocol messages (active).	No change.	0
G_{12}	Randomize passwords	No change.	$\frac{S}{ D }$

Fig. 14. Overview of proof steps in the proof of CAKE. Proof steps highlighted in grey require changes to consider quantum adversaries. These updated arguments are left as an open problem, but games G_2 , G_8 - G_{10} can likely be instantiated using SIM.

conditioned on values that were queried to the encryption oracles $\mathcal{E}_1$ and $\mathcal{E}_2$. Therefore, replacing the cipher with the simulator does not immediately result in a complete security proof. We therefore leave it as an open problem to construct such a proof

However, table 14 gives an overview of the game hops and sketches which games could potentially be instantiated with SIM.

Intuitively, the attack on CAKE resembles the one on OCAKE, by leveraging non-anonymity to venture a password guess (based on the transcript). The adversary then attacks the party, using its password guess. Note that for necessity, it is enough to present a specific attack against the protocol which the reduction then must use to solve some hard problem. This specifically means, we can consider a specific anonymity adversary that acts in a specific way when its inputs are not valid (i.e., the ciphertext was not produced under either key). Concretely, we give the attack, using an anonymity adversary $\mathcal{B}$ that rejects invalid inputs in Fig. 15. $\mathcal{B}$ that additionally outputs a reject symbol whenever neither public key matches. We showed for the analysis of OCAKE that for FO transformed schemes, it is possible to build such an adversary (Lemma 2). So, while there may be schemes where this attack is not feasible, showing the existence of the attack in one case is enough to show the necessity for a generic reduction. We

Algorithm Attack	
01	$P, P' \leftarrow P^2$ //Choose parties to attack
02	$e_1, e_2 \leftarrow \text{Execute}(P, P')$ //Get transcript of a session
03	For i in N
04	$pk_r \leftarrow \text{KGen}()$
05	for $pw \in \mathcal{D}$:
06	$pk \leftarrow \text{BC}^{-1}(pw, e_1)$
07	$c \leftarrow \text{BC}^{-1}(pw, e_2)$
08	$b \leftarrow \mathcal{B}(c, pk, pk_r)$ //Query anonymity adversary
09	if $b = 0 : \text{count}[pw] += 1$ //majority vote
10	$pw^* \leftarrow pw$ s.t. $\text{count}[pw]$ is maximized
11	$(pk^*, sk^*) \leftarrow \text{KGen}$ // Impersonation attack using candidate password
12	$e_1 \leftarrow \text{BC}(pw^*, pk^*)$
13	$e_2 \leftarrow \text{Send}_1(P', e_1)$
14	$c^* \leftarrow D_2(pw^*, e_2)$
15	$K^* \leftarrow \text{Decap}(sk^*, c^*)$
16	$SK \leftarrow \text{KDF}(\dots, K^*)$
17	$SK^* \leftarrow \text{Test}(P')$
18	if $SK = SK^*$ return 0 else return 1

Fig. 15. Attack Algorithm for adversary $\mathcal{A}$ using anonymity adversary $\mathcal{B}$ to attack security of CAKE. The attack first obtains a ciphertext, computes the the most likely password based on the anonymity adversary, and then actively attacks a session with that password.

now sketch an argument that the success probability of this attack is larger than that of a random password guess, proportionally to the advantage of the anonymity adversary $\mathcal{B}$. This argument is analogous to the one for OCAKE. This attack uses a majority vote strategy and amplification. Whenever pw matches the ‘correct’ password of the session, $\mathcal{B}$ is queried on a tuple (c, pk, pk_r) s.t. c was computed under pk , exactly as defined in the anonymity game. This triggers a vote for the correct password whenever $\mathcal{B}$ guesses correctly. Conversely, if the password was incorrect, the ciphertext is independent and $\mathcal{B}$ only responds with output 0 with some false-positive-rate. In conclusion, we get an offline dictionary attack against CAKE which then allows us to win the game via impersonation (lines 5ff).

B Additional figures

(O)CAKE security and BPR model

In this section, we include the security model, as well as a complete proof, restating previous work in sections that are indicated by a frame.

Classical BPR proof for OCAKE

In this section, we restate the model and preliminaries given in [AHH⁺23]. Our security analysis is based on the BPR model for authenticated key exchange [BPR00]: security of a protocol Π is modeled through a security experiment in

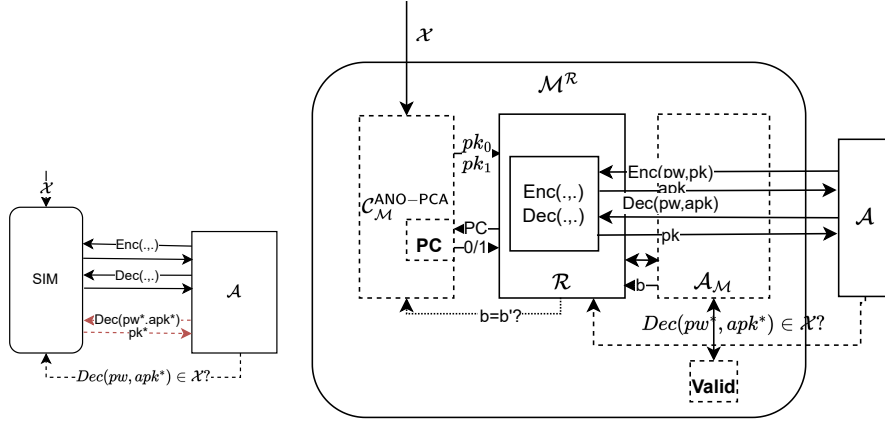


Fig. 16. Schematic overview of the meta-reduction $\mathcal{M}$ that turns a reduction $\mathcal{R}$ into a simulator according to SIM . The challenger additionally needs to provide plaintext checking on arbitrary ciphertexts, which is instantiated by brute-forcing the secret key and using a **Decap** oracle.

which the attacker interacts with oracles that represent honest parties (**Execute** and **Send**) as well as oracles that represent leakage of secret material (**Reveal** and **Corrupt**), and wins if it can distinguish an established session key from random. The involved oracles are described in more detail in Fig. 17. To exclude trivial attacks from consideration, [BPR00] define a freshness condition (Definition 24 below) that permits revealing a key on one side, and then testing the other (partnered) side, where partnered is defined as follows:

Definition 23 (Partnering). *Two instances $(P, i), (P', j)$ are partnered iff both instances have accepted (i.e. reached an **accept** instruction) with the same transcript and session key.*

Intuitively, ‘unfreshness’ expresses that the adversary may have learned the to-be-tested session’s key SK in a trivial way, i.e., by having interacted with the oracles revealing secret information in a way such that SK becomes trivially derivable regardless of the protocol’s nature. Concretely, the cases we cover in our freshness definition below are a), simply requesting the key from the **Reveal** oracle, and b), learning a password pw via **Corrupt** and then actively interfering with the test session, e.g., using pw to manipulate the peer into using a session key of the adversary’s choosing.

Definition 24 (Freshness with Forward Secrecy). *Suppose that the adversary made exactly one **Test** query, and it was to party P and instance i . We say session i of party P is **unfresh** if at any time, there was a **Reveal** query to instance (P, i) above or the instance (P', j) that it is partnered with. We also say the session is **unfresh** if both the following conditions hold:*

- Before the **Test** query, there was a **Corrupt** query on the test session’s holder P or its partnered peer P' .

- One of the messages sent to P concerning the test session was manipulated by the adversary, i.e., there was a $\text{Send}(P, i)$ query.

The session of (P, i) is only considered **fresh** if neither of these conditions are met.

The BPR security experiment $\text{Exp}_\Pi^{\text{BPR}}(\mathcal{A})$. The experiment is initialized by generating passwords between all involved parties P , according to the uniform distribution on password dictionary $\mathcal{D}$ of PAKE protocol Π . After that, adversary $\mathcal{A}$ is run with access to oracles **Execute**, **Send**, **Reveal**, **Corrupt** and **Test^b** described in Table 17. Oracle **Test^b** is defined relative to a randomly sampled bit b . $\mathcal{A}$ terminates by outputting a bit b' . The experiment returns 1 if $b' = b$, and otherwise 0.

Definition 25 (Key indistinguishability of PAKE). Let Π be a PAKE protocol. We say that an adversary $\mathcal{A}$, run in experiment $\text{Exp}_\Pi^{\text{BPR}}$, wins if it correctly guesses the bit according to which the test query was defined and if the **Test** query was issued for a party (P, i) that has terminated and is fresh (see Definition 24). We define the advantage of $\mathcal{A}$ against a PAKE protocol Π as

$$\text{Adv}_\Pi^{\text{BPR}}(\mathcal{A}) := |\Pr[\text{Exp}_\Pi^{\text{BPR}}(\mathcal{A}) \Rightarrow 1] - 1/2|.$$

Our modification of OCAKE uses key confirmation tags in both directions. While only the responder tag actually is needed for our security proof, we additionally include an initiator tag – following the ‘add client-to-server authentication’ (AddCSA) paradigm [BPR00] – to achieve explicit mutual authentication.

Definition 26 (Explicit Mutual Authentication). A protocol achieves explicit mutual authentication if parties accept if and only if there exists a partnered party that accepts with the same output.

Updated BPR proof for OCAKE

In this section, we fully flesh out our updated BPR security proof. Since we update the classical BPR proof, we work along the structure of that proof. For the reader’s convenience, we thus repeat Fig. 6 from the main body that summarizes the classical proof’s game-hops, which game-hops need updating, new proof techniques we use to do so, and the resulting loss.

Game	Description	Update?	Loss
G ₀	Original BPR game		
G ₁	Abort on KGen Collision.	Not necessary	$n_a^2 \cdot \eta_{\text{KGen}}$
G ₂	Abort on KDF Collision.	Not necessary	$\frac{ \mathcal{D} ^2}{ \mathcal{K}_{pw} }$
G ₃	IC lazy sampling w/ abort	This work (SIM)	-
G ₄	Prevent IC collisions	This work (SIM)	-
G ₅	Abort on Tag Collision.	This work (CO)	$\frac{80e^2 q_H^3 + 2}{ \mathcal{T} }$
G ₆	Sample IC using KGen	This work (SIM)	-
G ₇	Do not decrypt honest c .	Not necessary	$n_a \cdot \delta_{\text{KEM}}$
G ₈	Abort on tag under correct pw.	This work (eC0)	$\Pr[\text{corrPW}]$
G ₉	Make eC0 extractions on-line.	This work (eC0)	$n_a \cdot (n_s + q_H) \sqrt{2/ \mathcal{T} }$
G ₁₀	Randomize public key.	This work (SIM)	$\frac{1}{(1-\delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{ANO-PCA}}(\mathcal{B}^A) + \varepsilon_{\text{SIM}}(\mathcal{B}^A)$
G ₁₁	Randomize pre-key.	Not necessary	$\text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{IND-CPA}}(\mathcal{B}^I)$
G ₁₂	Move eC0 extractions to end.	This work (eC0)	$n_a \cdot (n_s + q_H) \sqrt{2/ \mathcal{T} }$
G ₁₃	Randomize Tag, Session Key.	This work (CO)	$\frac{q_H}{ \mathcal{K} }$
G ₁₅	Randomize Passwords.	Not necessary	0

In the following game hops, we will use the notational convention

$$\mathbf{Adv}_i := |\Pr[G_i(\mathcal{A}) \Rightarrow 1]| \text{ .}$$

We indicate game-hops that need no update (and thus are copy-pasted) by ‘restated’.

Game G₀: Original BPR game. PAKE Oracles are exactly as in existing proof (but included in the appendix for reference), however $\mathcal{H}$, KDF , KDF' and IC are quantumly accessible. **Game G₁: Collision Handling KEM key generation.** As previously, we have that $|\mathbf{Adv}_1 - \mathbf{Adv}_0| \leq n_a^2 \cdot \eta_{\text{KGen}}$.

BEGIN RESTATED

In this game, we abort whenever there are at least two sessions where the same ephemeral key pair (pk, sk) is sampled by the KEM key generation. Let η_{KGen} be the collision probability of KGen. Since games G₀ and G₁ are identical unless a collision occurs, we have that:

$$|\mathbf{Adv}_1 - \mathbf{Adv}_0| = \Pr[\text{KDFColl}] \leq n_a^2 \cdot \eta_{\text{KGen}}$$

END RESTATED

Game G_2 : Abort on KDF Collision. In this game, we abort whenever the key derivation function that maps passwords to block cipher keys outputs a collision on the password space. Since this bound is only dependent on the choice of KDF and dictionary $\mathcal{D}$, without any adversarial input, we do not have to model KDF as a random oracle and the bound is statistical. More precisely, if KDF is a function $f : \mathcal{D} \rightarrow \mathcal{K}_{pw}$ selected uniformly at random, the probability of it mapping any two inputs pw, pw' to the same output k_{pw} can be upper-bounded using a simple birthday collision bound: $|\mathbf{Adv}_2 - \mathbf{Adv}_1| \leq \Pr[\mathbf{KDFColl}] \leq \frac{|\mathcal{D}|^2}{|\mathcal{K}_{pw}|}$. **Game G_3 and G_4** are folded into game G_{10} .

Game G_5 : Abort on Tag Collision. One of the informal security goals of PAKE proofs is to rule out attacks where an adversary can *test* more than one password in a single online attack. One example of such an attack is the exploitation of a collision: if there should exist two passwords pw, pw' s.t. the responder tag tag computed from them for a given transcript is the same. Then the attacker could win the game by submitting tag without having uniquely identified the right password.

Therefore, in game G_5 , we abort whenever the attacker has found such a collision in H . To argue about the distance between G_5 and G_4 , we use the compressed oracle technique [CFHL21].

Let $\mathcal{A}$ be the adversary in the PAKE game. We define an adversary $\mathcal{B}$ in the collision-finding game that runs $\mathcal{A}$ and is successful if it outputs x, x' s.t. $H(x) = H(x')$ at the end of the game.

$\mathcal{B}$ simulates $\mathcal{A}$'s PAKE and random oracle queries. First, $\mathcal{B}$ replaces the random oracle with a compressed random oracle. The compressed random oracle allows $\mathcal{B}$ to record $\mathcal{A}$'s queries in a database.

Then, at the end of the game, $\mathcal{B}$ measures the database for collisions. Let R be the relation that describes a collision in the tag: for distinct x, x' and arbitrary sk, sk' we have that $H(x) \parallel sk = H(x') \parallel sk'$. Then we have $R \subseteq \mathcal{X}^l \times (\mathcal{T} \times \mathcal{SK})^l$ for $l = 2$.

$$\mathbf{CL} := \{D \mid \exists x \neq x' : D(x) = D(x') \neq \perp\}$$

Then, by Corollary 4.2 [CFHL21, Corollary 4.2], we have that for success probability in the original game p and success probability in the game using the compressed random oracle p' , we have that $\sqrt{p} \leq \sqrt{p'} + \sqrt{\frac{2}{|\mathcal{T}|}}$.

We now argue about the success probability p' , closely following the reasoning in section 2.3 of [CFHL21], (*Finding a Collision (with Parallel Queries)*). Let $\mathbf{SZ}_{\leq(s-1)}$ be a database of size at most $s - 1$. We set $k = 1$ as we do not model parallel queries, so we have using the transition probability of a database that does not contain a collision to one that *does*

$$\sqrt{p'} \leq \sum_{s=1}^q \llbracket \mathbf{SZ}_{\leq(s-1)} \setminus \mathbf{CL} \rightarrow \mathbf{CL} \rrbracket \leq \sum_{s=1}^q 2e \sqrt{10 \frac{q_H}{|\mathcal{T}|}} \leq 2qe \sqrt{10 \frac{q_H}{|\mathcal{T}|}}$$

where e is Euler's number. Therefore we conclude $p' \leq \frac{40q^3e^2}{|\mathcal{T}|}$. From $\sqrt{p} \leq \sqrt{p'} + \sqrt{\frac{2}{|\mathcal{T}|}}$ we then see by squaring that

$$p \leq p' + \frac{2}{|\mathcal{T}|} + \sqrt{\frac{2p'}{|\mathcal{T}|}} \leq \frac{40e^2(q^3 + \sqrt{2}q^{3/2}) + 2}{|\mathcal{T}|} \leq \frac{80e^2q^3 + 2}{|\mathcal{T}|}$$

Finally, this allows us to bound the distance:

$$|\mathbf{Adv}_5 - \mathbf{Adv}_4| \leq p \leq \frac{80e^2q_{\mathbb{H}}^3 + 2}{|\mathcal{T}|}.$$

labelgame:appendix.IC3**Game** G_6 is also folded into game G_{10} . **Game** G_7 : **Correctness.** As previously, $|\mathbf{Adv}_7 - \mathbf{Adv}_6| \leq n_a \cdot \delta_{\text{KEM}}$.

BEGIN RESTATED

In game G_7 , whenever there is a flow 2 query where the message was honestly generated by a matching session, we do not decapsulate to obtain the pre-key. Instead, if the message was generated by a matching session, we use the pre-key generated by that instance. Adversarially generated messages as well as ones that are forwarded from a non-matching session are decapsulated as before. This step is done in preparation for the reductions in the following two game hops. Games G_6 and G_7 are indistinguishable unless a correctness error occurred in game G_6 and therefore $|\Pr[\text{corrPW}_{G_6}] - \Pr[\text{corrPW}_{G_7}]| = |\mathbf{Adv}_7 - \mathbf{Adv}_6| = n_a \cdot \delta_{\text{KEM}}$.

END RESTATED

Game G_8 : **Abort on tag under correct pw.** We now abort whenever the adversary actively interacts with a session, meaning the session is not the execution of two honest parties and submits a tag that was generated using the correct password. This models the case that the attacker successfully guessed the password and used it to attack an honest part, which can happen both for

1. initiators, in which case the attacker sent a responder tag tag that uses the correct password, and for
2. responders, in which case the attacker sent an apk that uses the correct password.

Here we focus on case 1, since the other case is handled by simulation **SIM** (see 3). We need to adapt previous classical reasoning about this change to capture that oracle $\mathbb{H}$ now is quantum-accessible – previous reasoning exploited that game and reductions can observe the issued queries to $\mathbb{H}$, thus being able to immediately notice that tag was generated with the correct password. As a quantum counterpart to this approach, we use the extractable QROM formalism [DFMS22].

This models quantum-accessible random oracles like $\mathbb{H}$ as an extractable compressed random oracle **eC0** that has two interfaces, random oracle interface

$\mathbf{eCO.R0}$ and an extraction interface $\mathbf{eCO.E}_f$ that is defined relative to a function $f : X \times Y = T$, where X and Y are the random oracle's domain and co-domain, respectively, and T is some other set. For our purposes, we identify X with the domain of $\mathbf{H}$, so $X := \mathcal{K} \times \mathcal{D} \times \mathcal{PK}^2 \times \mathcal{C}$, and $Y := T := \mathcal{T} \times \mathcal{SK}$, the tag and session key space. Since we want to isolate the first part of the output, we choose the function f defined by the projection onto the tag space $f(\text{input}, y = (\text{tag}, K)) = \text{tag}$. Extraction interface $\mathbf{eCO.E}_f$ takes as input a classical value $t \in T$. It does a quantum analogue to going through the random oracle queries and returning an x such that $f(x, \mathbf{H}(x)) = t$: it performs suitable measurements that collapse the oracle database, just enough so that the classical procedure would yield one particular outcome x for all parts of the superposition. For our choice of f , $\mathbf{eCO.E}_f(\text{tag})$ simply returns a random oracle pre-image of $\text{tag}||sk$ for any sk .

Equipped with this extraction interface, the game waits until the attacker has finished and then calls $\mathbf{eCO.E}_f$ on all 'dishonest' tries, i.e., all tags tag that were received by an honest initiator whose password was not corrupted, without tag having been computed during a preceding honest execution of the respective responder. The game aborts if there exists a dishonest tag such that $\mathbf{eCO.E}_f(\text{tag})$ returns a pre-image (K, pw, apk, pk', c) such that (K, apk, pk', c) matches the values which the initiator computed during the session to which the tag belongs. Since we already ruled out random oracle collisions, this in particular means that pw is the correct password. We denote this event by corrPW . We note that the extraction queries themselves do not change $\mathcal{A}$'s view since they are performed only after $\mathcal{A}$ finished [DFMS22, Th. 4.3, item 1].

We can now conclude that

$$|\mathbf{Adv}_8 - \mathbf{Adv}_7| \leq \Pr[\text{corrPW}]_{G_8}.$$

Like in previous (classical) proofs, the change of probability for corrPW can be traced through the subsequent games G_8 - G_{14} , by building anonymity and indistinguishability reductions that define their output based on corrPW . **Game G_9 : Make $\mathbf{eCO}$ extractions online.** In this game, we switch to online extraction of the oracle used to derive the tag and session key. Instead of performing the extraction queries determining corrPW after running the attacker, we perform them already during the session runs. Whenever the attacker sends a tag tag to an honest initiator whose password was not corrupted, without tag having been computed during a preceding honest execution of the respective responder. This change is in preparation for the following two game hops, where the reductions need to detect certain edge-cases of adversarial input during the game.

To argue that this introduces little change, we use [DFMS22, Th. 4.3]: by properties 2.b and 2.c of Th. 4.3, any two subsequent queries to $\mathbf{eCO.R0}$ and $\mathbf{eCO.E}_f$ $8\sqrt{2\Gamma(f)/|\mathcal{K}| \cdot |\mathcal{T}|}$ -almost-commute, where

$$\Gamma(f) = \max_t \max_x |\{y \mid f(x, y) = t\}|, \quad (1)$$

which for our choice $f(x, (\text{tag}, K)) = \text{tag}$ equals $|\mathcal{K}|$. This means that we can commute the extraction queries into the game to right after when the respective

Send query was received. We need to commute at most $n_a \cdot (n_s + q_H H)$ many times until the extractions are at the right place, thus $|\mathbf{Adv}_9 - \mathbf{Adv}_8| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}$ and also $|\Pr[\text{corrPW}_{G_8}] - \Pr[\text{corrPW}_{G_9}]| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}$.

Game G_{10} : Randomize public-key pk . In this game, we randomize the public key used to create the responder ciphertext. First we recall the setting: in order to un-link the first (apk) and second (c, tag) protocol message, we argue that the ciphertext c does not leak which ciphertext was used to create it. More formally, we replace the public key used to derive c with a fresh uniformly random one, and reduce the distance in the games induced by this change to the ANO-PCA security of KEM.

We give a reduction that simulates the PAKE game for an adversary $\mathcal{A}$ that distinguishes BPR game G_{10} from G_8 and uses $\mathcal{A}$ to solve its ANO-PCA challenge. Consider the different query patterns that the reduction has to address. First we note that sessions that are passively attacked (queries to **Execute**) or those where the initiator is honest can always be simulated without the use of programming in the IC: honest parties simply use the challenge public keys in place of their honest keys.

The more involved case is the one where the responder is honest, but the initiator is not: this setting involves an *adversarially chosen* encrypted public key apk that is decrypted by the honest responder into a public key. In order to move to game G_{10} , where ciphertexts c are independent of the real public keys (or more accurately, the real password), the reduction reprograms decrypted public keys to challenge public keys in the ANO-PCA game.

We now show this proof step using only reprogramming according to the capabilities of **SIM** described above.

In a first step, the reduction answers all queries to **E** and **D** using the respective oracles of **SIM**. This change to **SIM** is not perfectly indistinguishable, therefore we have $\mathbf{Adv}_{10,1} \leq \varepsilon_{\text{SIM}}$ where ε_{SIM} by definition of **SIM**.

The challenge set $\mathcal{X}$ used to initiate **SIM** is equal to the first set of public keys $\{pk_{0,0}, pk_{0,1}, \dots, pk_{0,n_a}\}$ provided by the anonymity challenger. This is the set that indicates the ‘real’ public keys, while the set $\{pk_{1,0}, pk_{1,1}, \dots, pk_{1,n_a}\}$ indicates the independent keys used *after* the reduction. Now, whenever the adversary submits a value apk for a session associated with password pw and key $k \leftarrow \text{KDF}(pw)$, apk decrypts to a challenge public key whenever **SIM** programs successfully. For n_a actively attacked sessions, we have by definition of **SIM** that

$$\Pr[(\vec{apk}, \vec{sid}) \leftarrow \mathcal{A}^{\text{SIM}(\mathcal{X})} : \text{D}_{\text{SIM}}(k, apk_i) \in \mathcal{X} \forall (sid_i, apk_i) \in (\vec{sid}, \vec{apk}_i)] = 1 - \delta_{\text{SIM}}.$$

The reduction, on input apk , therefore decrypts apk to some challenge public key $pk_{0,j} \in \mathcal{X}$ and queries its challenger for a fresh anonymity challenge c related to that public key, and uses that in its response. That way, the adversary receives a ciphertext matching either pk_0 or pk_1 in each session, exactly as in the two games.

Finally, we note that since the reduction simulates all of $\mathcal{A}$ ’s oracles, so there is an edge case to consider: the case where $\mathcal{A}$ corrupts a party after it has queried

the Send^0 and Send^1 , but before it queries Send^2 for matching sessions while forwarding messages. In this case, the oracle Send^2 uses a challenge public key pk_0 that the attacker now *knows* due to the corruption. However, the reduction does not have the secret key associated with the public key, and it cannot decapsulate the ciphertext and recompute the tag. Therefore, when the oracle decides whether or not to reject an adversarially generated (c, tag) , the reduction has use the extraction interface of the extractable random oracle eCO to determine the pre-key that was used by the adversary. Then, the reduction can query its plaintext-checking oracle to decide if the message should be rejected or not. Since we switched to the extractable oracle in a previous game, this does not incur any additional loss here.

Finally, the reduction outputs 0 whenever $\mathcal{A}$ wins, and 1 else. Therefore, the reduction wins whenever $\mathcal{A}$ wins and SIM successfully programmed for all its queries and we have $\text{Adv}_{10,2} \leq \frac{1}{(1-\delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{\text{ANO-PCA}}$. We then arrive at the bounds

$$|\text{Adv}_{\text{G}_{10}} - \text{Adv}_{\text{G}_9}| \leq \text{Adv}_{10,1} + \text{Adv}_{10,2} \leq \frac{1}{(1-\delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{ANO-PCA}} + \varepsilon_{\text{SIM}}(\mathcal{B}^{\text{A}}).$$

$$|\Pr[\text{corrPW}_{\text{G}_{10}}] - \Pr[\text{corrPW}_{\text{G}_9}]| \leq \frac{1}{(1-\delta_{\text{SIM}})} \cdot \text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{ANO-PCA}} + \varepsilon_{\text{SIM}}(\mathcal{B}^{\text{A}}).$$

Game G_{11} : Randomize pre-key K . As previously, $|\text{Adv}_{11} - \text{Adv}_{10}| \leq \text{Adv}_{\text{KEM}}^{(n_s, n_a+1) - \text{IND-CPA}}(\mathcal{B}^{\text{I}})$.

BEGIN RESTATED

For all queries to the Send or Execute oracles where flag trivGuess is not raised before the query, we now randomize the pre-key K that is used to derive the final session key and the responder tag. For more details, see the pseudo-code in Figure 19. This change makes the pre-key independent of the ciphertext and the password for all fresh sessions. We now argue that an adversary noticing this change can be used to attack the indistinguishability property of the KEM. We define adversary $\mathcal{B}_0^{\text{I}}$ against the $(n, q_C) - \text{IND-CPA}$ experiment (defined in Fig. 3) as follows (for the sake of formality, we give the pseudo-code of $\mathcal{B}_0^{\text{I}}$ in Fig. 20):

$\mathcal{B}_0^{\text{I}}$ receives a vector of challenge public keys $\vec{pk}$, of dimension $n = n_s$ and can query its challenge oracle Chall , provided by its $(n, q_C) - \text{IND-CPA}$ challenger, at most $q_C = n_a + 1$ many times. $\mathcal{B}_0^{\text{I}}$ samples a challenge bit b' , runs $\mathcal{A}$ and answers $\mathcal{A}$'s queries to the Oracles H , E , KDF , Reveal , Send^0 , Send^2 , and $\text{Test}^{b'}$ according to the oracles in G_{10} .

On Send^1 queries, $\mathcal{B}_0^{\text{I}}$ issues a $\text{Chall}(j)$ query to its own challenger receive (c^*, K^*) , where j is the index of the public key which it uses to answer the query. If trivGuess has been raised, $\mathcal{B}_0^{\text{I}}$ generates a key pair and continues the protocol honestly without inserting any challenges in this session. If the same apk is submitted multiple times for sessions using the same password, the game is kept consistent by re-using the respective public key. When $\mathcal{A}$ outputs a guess

b , $\mathcal{B}_0^l$ checks if $b = b'$. In the case that $b = b'$, it returns 1 as its own output bit, otherwise, it returns 0.

$\mathcal{B}_0^l$ perfectly simulates G_{10} when run in the $(n_s, n_a + 1)$ – IND-CPA- game with challenge bit 0, G_{11} when run with challenge bit 1, and returns 1 if the adversary wins. Therefore, the difference between $\mathcal{A}$'s winning probabilities in games G_{10} and G_{11} is upper bounded by the respective $(n_s, n_a + 1)$ – IND-CPA advantage of $\mathcal{B}_0^l$ against KEM:

$$|\mathbf{Adv}_{11} - \mathbf{Adv}_{10}| \leq \mathbf{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{IND-CPA}}(\mathcal{B}_0^l)(\mathcal{B}_0^l)$$

To keep track of the change in the probability of $\Pr[\text{corrPW}]$, we can adapt the reduction $\mathcal{B}_0^l$ exactly like in the game-hop before by redefining the output bit to be 1 iff corrPW occurred and $|\Pr[\text{corrPW}_{G_{10}}] - \Pr[\text{corrPW}_{G_{11}}]| \leq \mathbf{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{IND-CPA}}(\mathcal{B}_0^l)(\mathcal{B}_0^l)$. At this point, pre-key K (for sessions between non-corrupted parties) is independent of the password and the protocol messages.

END RESTATED

Game G_{12} : Move eC0 extractions to end of game. In this game, we switch back to offline extraction, i.e., we only perform the extraction queries determining if a tag was valid after running the attacker. This is done in preparation for the next step that relies on measuring the compressed database at the end of the game. With the same reasoning as for game G_9 , we find

$$|\mathbf{Adv}_{12} - \mathbf{Adv}_{11}| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}$$

and

$$|\Pr[\text{corrPW}_{G_{11}}] - \Pr[\text{corrPW}_{G_{12}}]| \leq n_a \cdot (n_s + q_H) \sqrt{2/|\mathcal{T}|}.$$

Game G_{13} : Randomize Tag & Session Key. In the penultimate step of the proof, we randomize the responder tag tag and the session key SK , i.e., we simply sample random values from $\mathcal{T}$ and $\mathcal{SK}$, and return them as tag and session key. Recall, that in Game G_{12} the tag and session key are computed as $(tag, SK) \leftarrow \mathcal{H}(K_{\S}; pw, apk, pk', c)$ where $K_{\S}$ is a fresh uniformly random sample from $\mathcal{K}$ that is not used anywhere else in the protocol. Hence, the values in both games follow the same marginal distribution. However, in Game G_{12} they are consistent with the random oracle, while in Game G_{13} they are not.

Intuitively, an adversary can only notice this inconsistency in Game G_{13} if it ever learned the value of $\mathcal{H}$ at position $x^* = (K_{\S}; pw, apk, pk', c)$. Given that we simulate $\mathcal{H}$ using the compressed oracle technique, this means that the difference between this and the last game can be bounded by the probability that the final database D of the compressed oracle in Game G_{13} contains a value for x^* when measured in the end, i.e.,

$$|\mathbf{Adv}_{13} - \mathbf{Adv}_{12}| \leq \Pr[D(x^*) \neq \perp].$$

It remains to quantify $\Pr[D(x^*) \neq \perp]$. For this, note that in Game G_{13} , $K_{\S}$ is not used anywhere in the protocol anymore and therefore the whole view of the adversary is independent of $K_{\S}$. For that reason, we can delay the sampling of $K_{\S}$ until after we measured the database D . The database D has $q_{\mathbb{H}}$ entries. The probability that one of them is of the form $(K_{\S}; pw, apk, pk', c)$ for any value of $K_{\S}$ is therefore upper bounded by $1/|\mathcal{K}|$ and therefore

$$|\mathbf{Adv}_{13} - \mathbf{Adv}_{12}| \leq \Pr[D(x^*) \neq \perp] \leq \frac{q_{\mathbb{H}}}{|\mathcal{K}|}.$$

Game G_{14} : Randomize Passwords. It remains to upper-bound corrPW . In this game, the attacker's view is completely independent of the chosen passwords, up to corrupted ones. This means that we can replace all non-corrupted passwords with fresh ones at the end, after running the attacker, and defining event corrPW with respect to the resampled ones. We note that there exists exactly one pre-image (K, pw, apk, pk', c) per tag that could trigger corrPW , and that the simulation SIM does not raise this flag as it did in the previous proof.

We can now bound the probability of corrPW in game G_8 , using the number of send queries and the password distribution. Assuming a uniform distribution on a password dictionary of size $|\mathcal{D}|$, and upper-bounding $\mathcal{A}$'s number of send queries by n_a ,

$$\Pr[\text{corrPW}_{G_{14}}] \leq \frac{n_a}{|\mathcal{D}|}$$

And therefore $\Pr[\text{corrPW}_{G_8}] \leq \sum_{i=8}^{14} \Pr[\text{corrPW}_{G_i}]$.

Since the passwords and session keys are completely random from the adversary's view, we have that $|\Pr[G_{14}(\mathcal{A}) \Rightarrow 1]| = \frac{1}{2}$. We can finally sum up the terms and state the bound:

$$\begin{aligned} \mathbf{Adv}_{\text{OCAKE}}^{\text{BPR}}(\mathcal{A}) &\leq \sum_{i=0}^{14} |\Pr[G_i(\mathcal{A}) \Rightarrow 1]| \\ &\leq \frac{n_a}{|\mathcal{D}|} + n_a^2 \cdot \eta_{\text{KGen}} + \frac{|\mathcal{D}|^2}{|\mathcal{K}_{pw}|} + \frac{80e^2 q_{\mathbb{H}}^3 + 2}{|\mathcal{T}|} + n_a \cdot \delta_{\text{KEM}} + \\ &\quad + 4 \cdot n_a \cdot (n_s + q_{\mathbb{H}}) \sqrt{2/|\mathcal{T}|} + 2 \cdot \left(\frac{1}{(1 - \delta_{\text{SIM}})} \cdot \mathbf{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{ANO-PCA}} + \varepsilon_{\text{SIM}}(\mathcal{B}^{\text{A}}) \right) \\ &\quad + 2 \cdot \mathbf{Adv}_{\text{KEM}}^{(n_s, n_a + 1) - \text{IND-CPA}}(\mathcal{B}^{\text{I}}) + 2 \cdot \frac{q_{\mathbb{H}}}{|\mathcal{K}|} \end{aligned}$$

Notes on (O)CAKE security and the ideal cipher model

To give intuition for the needed properties of SIM , this section recollects the various generic attacks on PAKE protocols, how these attacks inflict conditions on OCAKE 's building blocks, and the proof techniques used to capture these conditions when idealizing the block cipher as an IC.

Generic attacks on (O)CAKE and their relation to the IC

We now recollect generic attacks on PAKE protocols, how they relate to OCAKE's building blocks, and the usage of the IC in security proofs.

Dictionary attacks and anonymity The first type of attack on PAKE protocols, including OCAKE, is a dictionary attack. In a dictionary attack, an attacker observes the protocol message exchanged between honest parties and attempts to extract information about the password (i.e., the long-term secret) from the protocol messages. As the password is what authenticates the honest parties, any successful dictionary attack which recovers the password results in a complete loss of security. For OCAKE (and CAKE), it is thus crucial that the second protocol message does not reveal the public key pk that was sent in the first message: knowing pk would allow to test password guesses, by encrypting pk under the different passwords and comparing the result against apk . For OCAKE, the second protocol message is a KEM ciphertext, it is thus natural to require that KEM satisfies anonymity, i.e., that KEM ciphertexts do not indicate to which public key they belong. Accordingly, the known security proofs utilize a reduction to the anonymity of KEM. This reduction has to distinguish ciphertexts belonging to proper public keys from ciphertexts belonging to random keys, and tricks the attacker into solving this task by programming its challenge public keys into suitable positions of the IC. This shows up, e.g., in game G_9 of the OCAKE security proof given in [AHH⁺23], and in games G_8 , G_9 and G_{10} of the CAKE security proof given in [PZ23].

In section 5 we show an attack on the protocol based on a lack of anonymity.

Dictionary attacks and public-key uniformity Another dictionary attack vulnerability appears for OCAKE (and CAKE) if the message apk does not decrypt to a valid public key under every password, as attackers can then rule out the passwords for which decrypting apk would yield an invalid public key. This can be captured by requiring that the KEM additionally satisfies public-key uniformity, i.e., that the public output parts of KGen are indistinguishable from random elements in the range of BC^{-1} . Similar to the anonymity reduction, the uniformity reduction will trick the attacker into solving the task at hand by programming its challenge public keys into suitable positions of the IC.

Password guessing attacks and extraction Since the passwords are assumed to be low-entropy, it is possible for attackers to guess the password and use their guess to attempt at a successful interaction with (or 'online' attack on) the honest party. Security proofs bound the probability of this event based on the dictionary size and leakage from previous sessions. This requires, however, that the involved security reductions can efficiently detect this event - the reductions are used to randomize certain outputs, but only if no password guess occurs, since in that case the randomization would be noticeable by the adversary. This means that we need a way to *extract* the password that is associated to the adversary's message.

Collision attacks The security of PAKE protocols significantly decreases if attackers can use a single interaction (or 'online' attack) with an honest party to

rule out several passwords from the (already comparably small) list of passwords. Accordingly, the next attack vector we consider are collision attacks, i.e., attacks that leverage the fact that an adversarially chosen protocol message would work for several distinct passwords. The proofs therefore will argue at some point that the attacker cannot come up with a message that could be used to rule out or confirm more than 1 password. (An equivalent requirement holds for UC PAKE). For OCAKE, an adversary could find an apk value s.th. apk is a valid encryption for two distinct passwords and public keys chosen by the adversary $(pw, pk), (pw', pk')$, i.e., such that we have $apk = \text{BC}(pw, pk) = \text{BC}(pw', pk')$, meaning the adversary might be able to try both passwords at once. Modeling BC as an IC and the derivation functions as random oracles, the proof of OCAKE accordingly eliminates/bounds the probability of certain collision events, as seen in games G_0 to G_5 [AHH⁺23]. Looking ahead, we will have to find equivalent counterparts to these arguments in the setting where the involved oracles are quantum-accessible. We have shown that the last remaining piece to the security proof of OCAKE considering quantum attackers is the reduction towards anonymity. In the classical proof, this step makes use of the ideal cipher model. However, when proving security against quantum attackers, we have to model the block cipher in a way that lets an attacker query it on arbitrary quantum states. To the best of our knowledge, there is currently no way to argue simulation of a quantumly accessible block cipher that allows for proof techniques such as reprogramming, lazy sampling of extraction in an equivalent way. Indeed, it may prove impossible to achieve these properties. However, it was never shown that using programmability is the only way to prove the statement. Hence, we are taking a step back and try and figure out the exact requirements on a simulator for the ideal cipher (and if we need one at all). To this end, we first provide more technical details and outline properties of the ideal cipher that play an important role in the existing proofs of both OCAKE (14) and CAKE (14). This analyses also allows us to determine some lower bounds on the parameters of SIM.

Role of the ideal cipher in the proof of OCAKE In this section, we recall the important role played by the ideal cipher in the proof of OCAKE that was given in [AHH⁺23]. Intuitively, the relevant proof steps have the following purpose: they unlink the first and second message in the protocol to prevent dictionary attacks on the password. In the proof of OCAKE, this is reflected as a reduction to the anonymity property of KEM, performed in game G_9 . We will now briefly describe the role of the ideal cipher in that part of the proof. In order to be successful, the anonymity reduction simulates the game to the attacker in a way that suitably places its challenges: it needs to make sure that (1) the ciphertexts used by the (simulated) responder are its challenge ciphertexts and (2) these ciphertexts are consistent with the protocol execution. It is convenient to argue separately about the case where the attack is on the responder or initiator side of the protocol. I.e., when the attacker is on the responder side, the reduction simulates the initiator oracle, and vice versa.

When simulating the initiator oracle, the reduction uses a challenge public key provided by the anonymity challenger and writes it into the ideal cipher simulation. This step is equivalent to querying IC encryption on that public key.

Conversely, when simulating a responder oracle, the reduction uses a challenge ciphertext whenever apk decrypts to one of its challenge public keys. An arising subtlety, however, is that the received apk might not have originated from the initiator oracle, but rather be a value that the attacker chose. This is problematic because the simulated responder needs to answer its query according to the honest decryption of this apk . Therefore, if the reduction wants to answer the query with a challenge ciphertext, the decryption must correspond to a challenge public key.

In the classical proof, the reduction simply answers all ideal cipher decryption queries (made by either adversary or game) with challenge public keys, unless the requested apk originates from a previous query in the forwards direction, i.e., a query to IC for some pk under the correct password of that session. This is shown schematically in Fig. 30. In consequence, D will return a challenge public key, thus circumventing the issue of not being able to output a challenge ciphertext in every session, unless the adversary made a forwards query with the correct password. In the proof of OCAKE, the probability of an attacker making such a forwards query is bounded by the probability of the attacker *guessing* the password of some session. There is one more subtlety to this step: in the case that public keys of KEM are not uniformly random elements of the message space $\mathcal{M}$ of the cipher, reprogramming outputs changes the output distribution. To address this issue, the proof adds a preliminary step that changes all decryption output samplings to be outputs of the key generation KGen in proof step G_6 . This step reduces the distance of the change to the public key uniformity PKU of KEM.

Take-away: exploited IC properties To summarize, the proof exploited the following properties of the ideal cipher model: a reduction simulating IC can

- observe and keep a list of all queries issued by the attacker, thereby being able to extract the information that a correct password guess occurred from the received apk ; and
- embed challenges into the ideal cipher outputs when it suits its goals, circumventing potential inconsistencies by using the query list to account for previous IC queries.

Role of the ideal cipher in the proof of CAKE We now also briefly describe the ways in which the CAKE proof given in [PZ23] relies on similar properties of the two involved ideal ciphers (E_1, D_1, E_2, D_2) . There are four game-hops that require embedding challenge values into the ideal ciphers' decryption outputs:

- Game G_2 : Decryption queries to the first cipher D_1 are sampled using KEM key generation (KEM fuzziness/Public-key uniformity)
- Game G_8 : $\mathcal{B}_4$ embeds challenge ciphertexts c into queries to D_2 (OW-rPCA security of KEM)

- Game G_9 : $\mathcal{B}_5$ embeds challenge public keys pk into queries to D_1 (OW-PCA security of KEM)
- Game G_{10} : $\mathcal{B}_6$ embeds challenge public keys pk into queries to D_1 (ANO-PCA security of KEM)

Two of these cases are conceptually equivalent to steps in the security proof for OCAKE. (Game G_2 for CAKE conceptually resembles game G_6 in the proof of OCAKE, and game G_{10} for CAKE conceptually resembles OCAKE game G_7). The other two games pose additional requirements: Game G_8 concerns the second ideal cipher that is used in the CAKE protocol, and does therefore not have a direct equivalent. The change in game G_9 of CAKE is conceptually equivalent to the change in game G_{10} of OCAKE. Notably though, this step requires programmability, whereas the equivalent step in OCAKE (G_{10} towards IND-CPA) does not due to its alternative proof structure. For OCAKE, the proof uses the anonymity of KEM before one-wayness or indistinguishability.

This difference also results in a difference in the requirements on KEM: OCAKE relies on the stronger notion of ANO-PCA security, while CAKE only requires weak anonymity. To summarize the difference, strong anonymity also holds for adversaries that have access to the key contained in the encapsulation. Since we aim to minimize the amount of programming done in the proof in this work, we will continue the analysis for the case of the OCAKE protocol.

This analyses also allows us to determine some lower bounds on the parameters of SIM.

Lower bounds on SIM parameters Now that we have shown that the simulation above is both necessary and sufficient to prove the security of OCAKE, we discuss its instantiation. When modeling assuming purely classical adversaries, we see that lazy sampling and reprogramming according to the ideal cipher model provides the necessary properties. As soon as we model quantum queries, these properties are no longer obvious. In this work we defined the minimum necessary properties as a first step to bridge the gap to a proof against quantum attackers. Since the new setting allows both quantum and classical attacks, we can use classical reasoning to define some lower bounds on the achievability of SIM.

We see that SIM is defined by two parameters, ε_{SIM} and δ_{SIM} . Informally, ε_{SIM} indicates the distance between the simulation and an ideal cipher, which can also be interpreted as the probability of distinguishing one from the other of an adversary interacting with one. An example would be the case where the simulation reprograms an already queried position: an adversary making two queries would detect this.

Parameter δ_{SIM} can be interpreted as the success probability of programming. This parameter will depend both on the instantiation of SIM as well as the setting where SIM is used: the adversary could output values that were the result of encryption queries. In those cases, the decrypted value is adversarially chosen (unless SIM reprograms the output). in the case of PAKE, this only happens whenever the attacker correctly guesses the password of a session, and we can upper-bound that probability. However, the probability of such an event still

gives a lower bound on δ_{SIM} and ε_{SIM} . Intuitively, either δ_{SIM} or ε_{SIM} or some combination of the two must be larger than the probability of this event. In the case of **OCAKE**, the relation takes the form $\varepsilon + \delta_{\text{SIM}} \geq \Pr[\text{corrPW}] = \frac{n_a}{|\mathcal{D}|}$ for n_a actively attacked sessions and a password dictionary $\mathcal{D}$.

Note that these lower bounds apply both in the classical and the quantum case.

Collisions in SIM and IC The final case to discuss is that of collisions: an ideal cipher $\text{IC} : \mathcal{D} \times \mathcal{M} \rightarrow \mathcal{M}$ always contains points s.t. for two passwords $(pw, pw') \in \mathcal{D}^2$ and two messages $(m, m') \in \mathcal{M}^2$, we have that $\text{E}(k_{pw}, m) = \text{E}(pw', m)$. An adversary that can find such collisions can more easily trigger the bad case where the decryption is outside of the challenge set. Even without knowing which pw^* is used to decrypt, an adversary outputting this apk doubles the probability of the decryption being one of $\mathcal{A}$'s chosen values. This increase also poses a lower bound on the simulation parameters. In the case of **OCAKE**, the relation takes the form $\varepsilon + \delta_{\text{SIM}} \geq \Pr[\text{corrPW}] + \Pr[\text{collision}]$ for n_a actively attacked sessions and a password dictionary $\mathcal{D}$.

Query	Return Value	Description
$\text{Execute}(P, i, P', j)$	(apk, c, tag)	Passive attack: Return transcript of an honest protocol execution between parties P and P' , using the i th/ j th session of P/P' .
$\text{Send}(P, i, msg, flow)$	msg'	Active attack: Send message msg to the oracle representing honest party P , causing it to proceed depending on its state. Flow indicator enumerates the messages in a run of the protocol and improves readability of the oracle.
$\text{Reveal}(P, i)$	$SK[P, i]/\perp$	Session key leakage: Return session key SK of (P, i) iff (P, i) terminated, else $\perp$; marks this instance and its matching instance "unfresh".
$\text{Corrupt}(P, PWD')$	$PWD[P, :]$	Password leakage or overwrite: Either return dictionary of passwords $PWD[P, :]$ held by party P , or allow adversary to overwrite password dictionary with $PWD'[P, :]$.
$\text{Test}^b(P, i)$	$SK[P, i]/SK^{\$}$	Session key challenge: Attack i th session of party P . Only for fresh, accepting instances. Returns either real or random session key depending on challenge bit b .
$\text{KDF}(pw)$	k_{pw}	Random oracle, input password $pw \in PWD$, output Ideal Cipher key k_{pw} .
$\text{H}(msg)$	tag, SK	Random oracle, input message msg , output $tag \in \mathcal{T}$ and $SK \in SK$.
$\text{E}(k, m)$	c	Ideal cipher encryption on input (key, message).
$\text{D}(k, c)$	m	Ideal cipher decryption on input (key, ciphertext).

Fig. 17. Overview of the PAKE adversary's oracles provided by the security game. Top part (above double midrule): oracles present in the BPR model. Bottom part: Random oracles and ideal cipher oracles to which the attacker additionally has access to when attacking the OCAKE protocol.

$\text{Send}_{G_6}^2(P, i, msg)$ 01 $c, tag \leftarrow msg$ 02 $K' \leftarrow \text{Decap}(sk, c)$ 03 04 $tag', SK \leftarrow \text{H}(pw, apk, pk, c, K', "i")$ 05 if $tag = tag'$: 06 $K[(P, i)] \leftarrow SK$ 07 else: return $\perp$	$\text{Send}_{G_7}^2(P, i, msg)$ if forward : $K' \leftarrow \text{responder's key } K$ else: $K' \leftarrow \text{Decap}(sk, c)$
--	--

Fig. 18. In game G_7 , whenever **forward** occurs (i.e., c is a matching responder's honest ciphertext) we use the responder's pre-key K instead of decapsulating c .

$\text{Send}_{G_{10}}^1(P, i, \text{msg})$	$\text{Send}_{G_{11}}^1(P, i, \text{msg})$
<pre> 01 $apk \leftarrow \text{msg}$ 02 $k_{pw} \leftarrow \text{KDF}(pw)$ 03 $pk' \leftarrow \text{D}(k_{pw}, apk)$ 04 if $\text{PK}[(k_{pw}, apk)] \neq \perp$: 05 $pk'_s \leftarrow \text{PK}[(k_{pw}, apk)]$ 06 else: 07 $(pk'_s, sk_s) \leftarrow \text{\\$KGen}$ 08 $\text{PK}[(k_{pw}, apk)] \leftarrow pk'_s$ 09 $(c, K) \leftarrow \text{\\$Encap}(pk'_s)$ 10 11 $tag, SK \leftarrow \text{H}(pw, apk, pk', c, K, "r")$ 12 return c, tag </pre>	<pre> $K_s \xleftarrow{\text{unif}} \mathcal{K}$ $tag, SK \leftarrow \text{H}(pw, apk, pk', c, K_s, "r")$ </pre>

Fig. 19. In game G_{11} , the pre-key set after querying **Send** or **Execute** is sampled independently of the password and the previous messages. Due to the change in game G_7 , this also randomizes the initiator side and we also write $K'_s \leftarrow K_s$.

Adversary $\mathcal{B}_0^I$	$\text{Send}^1(P, i, \text{msg})$
<pre> 01 input $\vec{pk}$ 02 $\text{pkIndex} = 0$ 03 $b \xleftarrow{\text{unif}} \{0, 1\}$ 04 $b' \leftarrow \mathcal{A}^{\mathcal{O}^b}(\vec{pk})$ 05 $b'_{\text{IND}} := [b = b']$ 06 output b'_{IND} </pre>	<pre> 07 $k_{pw} \leftarrow \text{KDF}(pw)$ 08 if $\exists$ record $\text{PK}[(k_{pw}, apk)]$: //handle replays 09 $pk'_s \leftarrow \text{PK}[(k_{pw}, apk)]$ 10 else: 11 $pk'_s \leftarrow \vec{pk}[\text{pkIndex}]$ 12 $\text{pkIndex} += 1$ 13 $\text{PK}[(k_{pw}, apk)] \leftarrow pk'_s$ 14 find j s.t. $pk'_s = \vec{pk}_j$ 15 $(c, K) \leftarrow \text{Chall}(j)$ 16 $tag, SK \leftarrow \text{H}(pw, apk, pk', c, K, "r")$ 17 return c, tag </pre>

Fig. 20. (n, q_C) – IND-CPA adversary $\mathcal{B}_0^I$, used to reason about the hop from game G_{10} to G_{11} . The set of oracles is $\mathcal{O} = \{\text{KDF}, \text{E}, \text{D}, \text{Execute}, \text{Send}, \text{Reveal}, \text{Corrupt}\}$.

Initialisation 01 for $(P, P') \in \mathcal{P} \times \mathcal{P}$ 02 $pw \leftarrow \\$PW()$ 03 $PWD[\{P, P'\}] \leftarrow pw$ Execute(P, i, P', j) 04 $apk \leftarrow \text{Send}^0(P, i, \perp)$ 05 $c, tag \leftarrow \text{Send}^1(P', j, apk)$ 06 $\text{Send}^2(P, i, (c, tag))$ 07 $\text{MANIP}[\{P, : \}] \leftarrow \text{false}$ 08 return (apk, c, tag) Corrupt(P, PWD') 09 $PWD_P \leftarrow PWD[\{P, : \}]$ 10 $\text{CRPT}[\{P, : \}] \leftarrow \text{true}$ 11 $PWD[\{P, : \}] \leftarrow PWD'[\{P, : \}]$ 12 return PWD_P Reveal(P, i) 13 $\text{RVL}[(P, i)] \leftarrow \text{true}$ 14 return $\text{SK}[(P, i)]$ Test$^b(P, i)$ 15 $SK_0 \leftarrow K[(P, i)]$ 16 $SK_1 \xleftarrow{\text{unif}} \\SK 17 if $(\text{CRPT}[\{P, P'\}] \text{ and } \text{MANIP}[(P, i)])$ 18 or if $(\text{RVL}[(P, i)] \text{ or } \text{RVL}[(P', j)])$ 19 or if $(K[(P, i)] = \perp)$: return $\perp$ 20 else: return SK_b	Send$^0(P, i, msg)$ 21 $\text{MANIP}[(P, i)] \leftarrow \text{true}$ 22 $k_{pw} \leftarrow \text{KDF}(pw)$ 23 $(pk, sk) \leftarrow \\$KGen$ 24 $apk \leftarrow E(k_{pw}, pk)$ 25 return apk Send$^1(P, i, msg)$ 26 $\text{MANIP}[(P, i)] \leftarrow \text{true}$ 27 $apk \xleftarrow{\text{parse}} msg$ 28 $k_{pw} \leftarrow \text{KDF}(pw)$ 29 $pk' \leftarrow D(k_{pw}, apk)$ 30 $(c, K) \leftarrow \\$Encap(pk')$ 31 $tag, SK \leftarrow H(K, pw, apk, pk', c)$ 32 $\text{SK}[(P, i)] \leftarrow SK$ 33 return (c, tag) Send$^2(P, i, msg)$ 34 $\text{MANIP}[(P, i)] \leftarrow \text{true}$ 35 $c, tag \xleftarrow{\text{parse}} msg$ 36 $K' \leftarrow \text{Decap}(sk, c)$ 37 $tag', SK' \leftarrow H(K'; pw, apk, pk, c)$ 38 if $tag' = tag$ 39 $\text{SK}[(P, i)] \leftarrow SK$
--	---

Fig. 21. The oracles in the security game for OCAKE. PWD is the dictionary of the parties' passwords and CRPT, MANIP and RVL indicate the corruption status of a session. Password generation in the initialization phase (**Initialization**) is modeled using the long-lived key generator PW .

Initialisation 01 for $(P, P') \in \mathcal{P} \times \mathcal{P}$ 02 $pw \leftarrow \\$PW()$ 03 $PWD[\{P, P'\}] \leftarrow pw$ 04 $PK \leftarrow \emptyset$	Send$^0(P, i, msg)$ 05 $\text{MANIP}[(P, i)] \leftarrow \text{true}$ 06 $k_{pw} \leftarrow \text{KDF}(pw)$ 07 $(pk, sk) \leftarrow \\$KGen$ 08 if $pk \in PK$ abort 09 $PK = PK \cup \{pk\}$ 10 $apk \leftarrow E(k_{pw}, pk)$ 11 return apk
--	--

Fig. 22. The game change for game G_1 Abort on KGen Collision.

Initialisation	Send⁰(P, i, msg)
01 for (P, P') ∈ $\mathcal{P} \times \mathcal{P}$	14 MANIP[(P, i)] ← true
02 pw ← \$ PW()	15 $k_{pw} \leftarrow \text{KDF}(pw)$
03 PWD[{P, P'}] ← pw	16 if $k_{pw} \in \mathcal{L}_{pw}$ abort
04 PK ← ∅	17 $\mathcal{L}_{pw} = \mathcal{L}_{pw} \cup \{k_{pw}\}$
05 $\mathcal{L}_{pw} \leftarrow \emptyset$	18 (pk, sk) ← \$ KGen
Send¹(P, i, msg)	19 if pk ∈ PK abort
06 MANIP[(P, i)] ← true	20 PK = PK ∪ {pk}
07 $apk \xleftarrow{\text{parse}} msg$	21 $apk \leftarrow \text{E}(k_{pw}, pk)$
08 $k_{pw} \leftarrow \text{KDF}(pw)$	22 return apk
09 $pk' \leftarrow \text{D}(k_{pw}, apk)$	
10 (c, K) ← \$ Encap(pk')	
11 tag, SK ← H(K, pw, apk, pk', c)	
12 SK[(P, i)] ← SK	
13 return (c, tag)	

Fig. 23. The game change for game G₂ Abort on KDF Collision.

Initialisation	Send²(P, i, msg)
01 for (P, P') ∈ $\mathcal{P} \times \mathcal{P}$	17 MANIP[(P, i)] ← true
02 pw ← \$ PW()	18 c, tag $\xleftarrow{\text{parse}} msg$
03 PWD[{P, P'}] ← pw	19 $K' \leftarrow \text{Decap}(sk, c)$
04 PK ← ∅	20 if $c \in \mathcal{C} : K' \leftarrow K$
05 $\mathcal{L}_{pw} \leftarrow \emptyset$	21 tag', SK' ← H(K'; pw, apk, pk, c)
06 $\mathcal{C} \leftarrow \emptyset$	22 if tag' = tag
Send¹(P, i, msg)	23 SK[(P, i)] ← SK
07 MANIP[(P, i)] ← true	
08 $apk \xleftarrow{\text{parse}} msg$	
09 $k_{pw} \leftarrow \text{KDF}(pw)$	
10 $pk' \leftarrow \text{D}(k_{pw}, apk)$	
11 (c, K) ← \$ Encap(pk')	
12 $\mathcal{C} \leftarrow \mathcal{C} \cup c$	
13 tag, SK ← H(K, pw, apk, pk', c)	
14 TAG ← TAG ∪ tag	
15 SK[(P, i)] ← SK	
16 return (c, tag)	

Fig. 24. The game change for game G₇ Do not decrypt honest c. Set the responder's pre-key to the matching initiator's.

<u>Initialisation</u>	<u>Send²(P, i, msg)</u>
01 for (P, P') ∈ P × P	08 MANIP[(P, i)] ← true
02 pw ← \$PW()	09 c, tag ← ^{parse} msg
03 PWD[{P, P'}] ← pw	10 K' ← K
04 PK ← ∅	11 tag', SK' ← H(K', pw, apk, pk, c)
05 L _{pw} ← ∅	12 if tag' = tag
06 C ← ∅	13 if not tag ∈ TAG and not CRPT[{P, P'}] abort
07 TAG ← ∅	14 SK[(P, i)] ← SK

Fig. 25. The game change for game G₈ Abort on correct password. Abort if we receive a tag that is correct unless it is honest or the password was corrupted.

<u>Send¹(P, i, msg)</u>
01 MANIP[(P, i)] ← true
02 apk ← ^{parse} msg
03 k _{pw} ← KDF(pw)
04 pk' ← D(k _{pw} , apk) pk _{\$} ← PK
05 (c, K) ← \$Encap(pk') (c, K) ← \$Encap(pk' _{\$})
06 C ← C ∪ c
07 tag, SK ← H(K, pw, apk, pk', c)
08 TAG ← TAG ∪ tag
09 SK[(P, i)] ← SK
10 return (c, tag)

Fig. 26. The game change for game G₁₀ Anonymity.

<u>Send¹(P, i, msg)</u>
01 MANIP[(P, i)] ← true
02 apk ← ^{parse} msg
03 k _{pw} ← KDF(pw)
04 pk _{\$} ← PK
05 (c, K) ← \$Encap(pk' _{\$})
06 K _{\$} ← \$K
07 C ← C ∪ c
08 tag, SK ← H(K, pw, apk, pk', c) tag, SK ← H(K _{\$} , pw, apk, pk', c)
09 TAG ← TAG ∪ tag
10 SK[(P, i)] ← SK
11 return (c, tag)

Fig. 27. The game change for game G₁₁ Indistinguishability.

```

Send1( $P, i, msg$ )
01  $MANIP[(P, i)] \leftarrow \text{true}$ 
02  $apk \xleftarrow{parse} msg$ 
03  $k_{pw} \leftarrow \text{KDF}(pw)$ 
04  $pk_{\S} \leftarrow \mathcal{PK}$ 
05  $(c, K) \leftarrow \S \text{Encap}(pk'_{\S})$ 
06  $C \leftarrow C \cup c$ 
07  $K_{\S} \leftarrow \S \mathcal{K}$ 
08  $tag, SK \leftarrow H(K_{\S}, pw, apk, pk', c)$ 
09  $TAG \leftarrow TAG \cup tag$ 
10  $tag_{\S} \leftarrow \S \mathcal{T}, SK_{\S} \leftarrow \S SK$ 
11  $SK[(P, i)] \leftarrow SK$   $SK[(P, i)] \leftarrow SK_{\S}$ 
12 return ( $c, tag$ ) return ( $c, tag_{\S}$ )

```

Fig. 28. The game change for game G_{13} Randomize Tag and Session key.

```

Algorithm Attack
13  $P, P' \leftarrow \mathcal{P}^2$  // Choose parties to attack
14  $apk \leftarrow \S \mathcal{PK}$ 
15  $(c^*, tag^*) \leftarrow \text{Send}^1(P', apk^*)$ 
16
17 for  $pw$  in  $\mathcal{D}$  // Obtain candidate public keys
18    $pk[pw] \leftarrow \text{BC}^{-1}(pw, apk)$ 
19 for  $pw \in \mathcal{D}$  // Compute pairwise guess
20    $count\_wins[pw] = 0$ 
21   for  $(pw, pw') \in \mathcal{D}^2$ 
22      $winner \leftarrow \mathcal{B}(c, pk[pw], pk[pw'])$ 
23     if  $winner == 0$ 
24        $count\_wins[pw] + = 1$ 
25     else //  $winner = 1$ 
26        $count\_wins[pw'] + = 1$ 
27  $pw^* \leftarrow \perp$  // Compute best guess
28  $max\_count \leftarrow 0$ 
29 for  $pw \in \mathcal{D}$ 
30   if  $count\_wins[pw] \geq max\_count$ 
31      $max\_count = count\_wins[pw]$ 
32      $pw^* \leftarrow pw$ 
33
34  $(pk^*, sk^*) \leftarrow \text{KGen}$  // Attack using candidate password
35  $apk^* \leftarrow \text{BC}(pw^*, pk^*)$ 
36  $(c^*, tag^*) \leftarrow \text{Send}^1(P', apk^*)$ 
37  $K^* \leftarrow \text{Decap}(sk^*, c^*)$ 
38  $SK \leftarrow \text{KDF}(\dots, K^*)$ 
39  $SK^* \leftarrow \text{Test}(P')$ 
40 if  $SK == SK^*$  return 0
41 else return 1

```

Fig. 29. Attack Algorithm for adversary $\mathcal{A}$ using anonymity adversary $\mathcal{B}$ to attack security of OCAKE. The attack uses a single transcript to compute the most likely password, then actively attacks a session with that password. Lines 7 to 21 are used to find the best possible guess out of all passwords using an anonymity adversary that works on pairs of public keys.

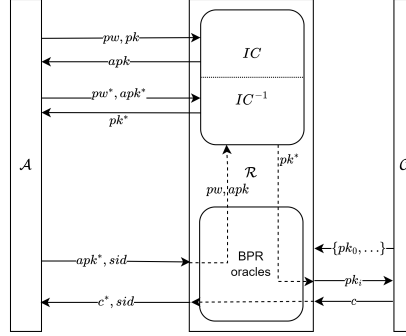


Fig. 30. Schematic view of anonymity reduction $\mathcal{R}$, used to decouple KEM ciphertexts from their originating public keys. $\mathcal{R}$ receives challenge public keys and ciphertexts from ANO-PCA challenger $\mathcal{C}$ and interacts with the PAKE adversary $\mathcal{A}$ to solve its challenges. $\mathcal{R}$ provides the PAKE BPR oracles and the ideal cipher oracles. When receiving a message apk from $\mathcal{A}$ whose decryption pk^* equals a challenge public key pk_i , $\mathcal{R}$ responds with an anonymity challenge c . $\mathcal{A}$ can make arbitrary quantum queries to IC at any time. (shown in red).

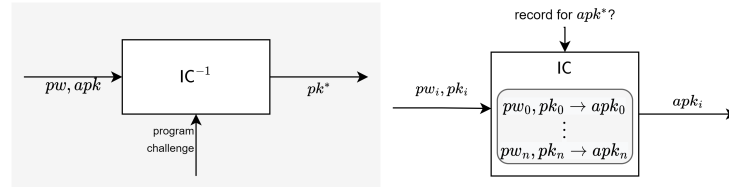


Fig. 31. Proof techniques used in the existing OCAKE proof: (r) In order to insert its anonymity challenges into the communication, the anonymity reduction sets all IC decryption outputs to challenge public keys. (l) Identifying correct password guesses (raising flag `corrPW`) requires checking if an (adversarially chosen) apk resulted from an (adversarial) IC encryption query that used the correct password.